
Technical Whitepaper · TECHNICAL WHITEPAPER

Aelion Twelve (A12)

Super-Orchestration Cloud Platform

Technical Whitepaper

Version 1.0 (Initial Release · Public Release Edition) · September 2026

Trusted Access · Intelligent Scheduling · Layered Value Chain for Heterogeneous GPU Computing Power
CaaS Computing Power → MaaS Model Services → AaaS Agent Applications

A12 Project | Version 1.0 (Initial Release · Public Release Edition) | September 2026

System Architecture · Trusted Verification · Intelligent Scheduling · Model Services · Agent Platform · Token Economics and Roadmap

AELION TWELVE (A12) · TECHNICAL WHITEPAPER

[Abstract](#)

[Chapter 1 Industry Background and Problem Definition](#)

[Chapter 2 Project Positioning and Governance Transparency](#)

[Chapter 3 System Architecture Overview](#)

[Chapter 4 Layer 1: Resource Access and Trusted Verification](#)

[Chapter 5 Layer 2: Scheduling and Resource Pooling](#)

[Chapter 6 Layer 3: Model Services \(MaaS\)](#)

[Chapter 7 Layer 4: Agents and Applications \(AaaS\)](#)

[Chapter 8 Token Economic Model](#)

[Chapter 9 Technology Roadmap](#)

[Chapter 10 Risk Factors and Disclaimer](#)

[Appendix A Glossary](#)

[Appendix B Index of Figures and Tables](#)

[Appendix C Figure Parameter Tables \(Figure 5-1 / Figure 6-1\)](#)

[References](#)

[Abstract](#)

§0 Version Effectiveness Statement: This document is the Aelion Twelve (A12) Super-Orchestration Cloud Platform Technical Whitepaper, Version 1.0 (Initial Release · Public Release Edition), published in September 2026. This edition is finalized on the principle of "including only content currently supported by corroborating materials": items not yet obtained—such as patent application numbers, full third-party audit reports, and public corroboration of investors—are not included in this edition; once the relevant documents are available, they will be disclosed through the official website's disclosure channels and incorporated in subsequent version updates. The version identifiers on the cover and in the page header are abbreviations of this statement; in case of any discrepancy, this statement prevails.

A12 is a scheduling and service platform for heterogeneous GPU computing resources. It provides unified access, verification, metering, and scheduling for cloud GPU clusters, professional IDCs, and idle workstation/edge GPUs. Based on task characteristics, it keeps high-communication-intensity large-model training on high-speed interconnected centralized clusters, distributes low-communication-intensity inference, retrieval-augmented generation (RAG), parameter-efficient fine-tuning (LoRA/QLoRA), and batch-processing tasks to more cost-effective distributed resources, and provides model services (MaaS), agent applications (AaaS), and a complete developer toolchain on top of a unified task interface.

This whitepaper describes only designs that have been verified to date or that have a clear engineering path, and follows three writing disciplines: all performance metrics are design targets rather than measured conclusions; all security mechanisms are accompanied by a statement of their boundaries and failure conditions; and all information concerning the project entity and its history is presented in the form of a disclosure list, which shall not serve as a basis for credibility prior to disclosure.

Keywords: heterogeneous computing-power scheduling; trusted resource verification; hybrid computing-power supply; model services; agent applications

Instance Cold Start	Time to First Token TTFT	GPU Effective Payload	Validation Pass Rate
$\leq 10\text{ s}$ Containerized Instance Scheduling • Design Target	$\leq 400\text{ ms}$ Public Beta Median • Design Target	$\geq 75\%$ Continuous Batching • Design Target	$\geq 99\%$ Internal Beta Exit Criterion • Go/No-Go

Chapter 1 Industry Background and Problem Definition

1.1 The Structural Status Quo of AI Computing-Power Supply

Generative AI workloads are rapidly diverging: foundation-model pre-training and full-parameter fine-tuning require high-frequency gradient synchronization and inherently depend on NVLink/InfiniBand-class cluster interconnects; whereas tasks such as inference, embedding, rerank, RAG data processing, and LoRA/QLoRA fine-tuning exhibit communication intensity lower by one to several orders of magnitude and can be completed with closure within a single machine or a small-scale cluster. The two types of workloads differ significantly in their infrastructure requirements, yet have long been served by "one and the same supply model."

At the same time, the supply side of computing power exhibits a three-tier structure: hyperscale cloud platforms offer standardized but relatively expensive managed computing power; professional AI IDCs and enterprise-owned clusters possess high-speed interconnects but suffer from large utilization fluctuations; and tens of millions of consumer-grade and workstation-grade GPUs worldwide remain idle for lack of a trusted access and scheduling mechanism (the order of magnitude is an estimated reference based on publicly available industry figures, not an exact statistic; all industry data in this chapter are used to illustrate structural judgments). The problem is not "insufficient computing power," but that computing power with different physical conditions has not been organized onto the appropriate workloads.

1.2 The Organizational Efficiency Problem of Hybrid Supply

Unifying the orchestration of the above three tiers of supply requires solving four engineering problems:

1. **Heterogeneous adaptation: differences in drivers, runtimes, and operator libraries across NVIDIA, AMD, and other accelerator cards make cost-free migration of upper-layer tasks difficult. Any solution claiming "completely imperceptible unification" sidesteps the real adaptation workload.**
2. **Resource trustworthiness: in an open-access environment, resource spoofing, VRAM overselling, and duplicate declarations are possible. Relying solely on node self-reported information is insufficient to establish the trust required for scheduling.**
3. **The tension between SLA and elasticity: idle edge GPUs have unstable online rates, and directly carrying enterprise-grade online services on them would cause availability collapse; whereas relying entirely on centralized resources forgoes the utilization of idle computing power.**
4. **Economic sustainability: a model that subsidizes supply solely through token emissions falls into a cycle of "emission — selling pressure — incentive failure" when real demand is insufficient.**

1.3 Market Space and A12's Business Value

Market size: AI infrastructure is undergoing historic expansion. According to IDC, global AI infrastructure (hardware) spending reached approximately USD 318 billion in 2025 (+107.6% YoY), is projected to reach USD 497 billion in 2026, and will exceed USD 1 trillion by 2029 (IDC Quarterly AI Infrastructure Tracker, 2026-07); Gartner projects

total global AI spending (including software and services) of USD 2.52 trillion in 2026 (Gartner, 2026-01). NVIDIA's data-center business recorded FY2026 (ended January 2026) revenue of USD 193.7 billion (+68% YoY), about 90% of its total revenue (NVIDIA official financial report, 2026-02)—compute has become the most value-concentrated segment of the AI industry chain.

The mismatch exists on both sides simultaneously. On one end, high-end compute remains persistently scarce: Barclays estimates that inference demand in 2026 will reach $4.5 \times$ training demand, implying a chip capital-expenditure gap of nearly USD 300 billion (Barclays, 2026); McKinsey projects that about USD 6.7 trillion of data-center investment will be needed globally by 2030, and the four largest cloud providers' combined 2026 capex of about USD 725 billion (+77% YoY) remains insufficient to cover it (McKinsey, 2026-03). On the other end, existing compute faces falling prices and idle capacity: on-demand H100 rental prices have fallen from their 2023–2024 peak of USD 8–10/hour to about USD 2–3/hour as of September 2026, the A100 is already below USD 1/hour, and the RTX 4090 runs at about USD 0.2–0.55/hour (third-party rental market data, 2026-03/09); industry estimates suggest that roughly 200 million consumer-grade GPUs worldwide sit idle (Blockworks Research, 2024, estimated).

Workload structure is also inverting: Deloitte estimates that inference already accounted for about half of AI compute consumption in 2025 and will reach two-thirds in 2026, trending long-term toward an "80% inference / 20% training" structure (Deloitte, 2025-11). Medium- and low-communication-intensity workloads such as inference, RAG, embedding, batch processing, and parameter-efficient fine-tuning are precisely what distributed compute can carry—meaning the addressable market of distributed compute does not shrink with the arms race for high-end training compute, but instead keeps expanding as inference workloads explode.

The capitalization levels of comparable projects provide a direct reference for A12. As of mid-September 2026, the circulating market caps of major projects in the DePIN compute track were approximately: Render (RENDER) ~USD 720 million; Akash (AKT) ~USD 150 million; Aethir (ATH) ~USD 96 million, with the project's self-reported annualized revenue (ARR) exceeding USD 156 million and a network covering 430,000+ GPU containers; io.net (IO) ~USD 49 million (sources: CoinMarketCap, CoinGlass, 2026-09; crypto-asset prices are highly volatile, and the figures above are point-in-time snapshots only).

Projecting A12's value space from this: using Aethir's self-reported ARR (~USD 156 million) as the reference, even under the conservative scenario of one-tenth of it (~USD 15 million annualized revenue), against the track's current "market cap / revenue" range of roughly $3-8 \times$, the corresponding platform value is on the order of USD 50–120 million; if a network effect at Render's scale were achieved, the corresponding market-cap space would exceed USD 500 million. This projection is a magnitude illustration based on public comparable data, intended to help readers build intuition about "track position—revenue—valuation," and does not constitute any promise of returns or price prediction.

In summary: the structural mismatch of high-end scarcity coexisting with idle stock, the migration of workload structure toward inference, and the already-validated capitalization path of the DePIN track together constitute A12's market opportunity window. Whether A12 can realize this value depends on its execution capability in organizing idle and underutilized compute into sellable services—precisely the goal that the technical design in Chapters 3–7 is built to support, and the engineering system corresponding to the solution outlined in 1.4.

1.4 A12's Solution (Overview)

In response to the above problems, A12's solution can be summarized in four points, whose feasibility boundaries are explained one by one in the corresponding chapters:

1. **A scheduling architecture that diverts traffic by communication intensity: high-communication tasks are fixed in centralized clusters, while low-communication tasks are pushed down to distributed resources (Chapters 3 and 5).**
2. **A layered trusted resource verification system: trust is established through a combination of "declaration verification + measured challenges + runtime telemetry + task-level verification + historical reputation," without relying on any single "absolutely tamper-proof" mechanism (Chapter 4).**
3. **A dual-track supply model: the elastic best-effort mode carries interruptible tasks, while the dedicated guaranteed mode carries SLA business, and the two are physically isolated (Chapters 4 and 5).**
4. **An economic model anchored in real service revenue: platform revenue comes first, tokens serve as incentive and settlement instruments, and buyback-and-burn is linked to revenue with a hard cap (Chapter 8).**

Chapter 2 Project Positioning and Governance Transparency

2.1 Platform Positioning Statement

A12 adopts an architectural positioning of "centralized operation + distributed execution": the Operating Entity centrally handles resource admission verification, task scheduling, billing and settlement, dispute resolution, and security risk control; the computing resources themselves are distributed globally (individuals, institutions, IDCs, and edge nodes), task execution is completed in a distributed manner at the location of the resources, and scheduling decisions are made based on the real-time status of each node. Users choose A12 because the platform reduces the organizational cost of using distributed heterogeneous computing power through a unified interface and trusted verification.

Platform commitments: all scheduling and billing rules are publicly available for inspection; task metering records are exportable and auditable by tenants; if on-chain attestation and settlement are introduced in the future, they will serve only as an optional protocol layer to enhance transparency (see §8.6) and will not change the platform's operating responsibility entity.

2.2 Project Entity and Team Disclosure (Mandatory Disclosure List)

Disclosure status: the information in the table below is compiled based on the Operating Entity's external disclosure materials and public registration records. The official email, office address, legal contact channel, and official social accounts were published on the official website's legal page and footer on 2026-09-15; The official primary domain is pending release; and beta access uses the backup domain a120.a12ai.com/A120/ before its official activation. Subsequent disclosures (patent application numbers, the full CertiK audit report, public corroboration of investors, etc.) will be released through the official website's disclosure page, after the primary domain is activated).

Disclosure Item	Content	Status
Operating Entity	Aelion Twelve Limited, registered with Corporations Canada, registration number 1819483-1 , has obtained the Certificate of Incorporation	Provided (registration information is publicly verifiable)
Regulatory license	MSB Registration Number: 31000340957743	Provided (registration information

is publicly verifiable)

Patent Application	Nigerian Invention Patent Application No.: F/PT/NC/O/2026/25182	Provided (the application number is publicly verifiable)
Core Team	Greg Herlihy: Chief Technology Officer, 9 Years at Microsoft -Software Development Lead, Engineer at Meta Headquarters, 8 Years at Skillz Inc. (NYSE-listed) - Core Early Team Member, Contributor to HTTP/1.1 & MHTML Standards, Chief Architect at Kekosoft Adrian Lin : Chief Distributed Computing Scientist, Silicon Valley-based Chinese engineer, 12+ years of experience in distributed computing and resource scheduling; Tunde Akinwale: Chief AI Infrastructure Architect, 10+ years of experience in AI infrastructure and cloud-native architecture; Ethan Carter : AI Platform Engineering Lead, 10+ years of experience in large-scale internet and AI platform engineering; Amara Okoye : GPU Infrastructure and Observability Lead, 7+ years of experience in GPU/Cloud Infrastructure; Noah Bennett : ML Systems and Agent Research Lead	Roster provided
Contact Information	Official email: Cs001@a12ai.com; legal contact channel: aelion12ai@gmail.com (shared with the official email); mailing address for legal documents: 2482 Yonge St, Suite 185, Toronto ON, Canada	Published (official website legal page and footer, 2026-09-15)
Code Openness	Core scheduling and verification module code repository: https://github.com/aelion-twelve; the third-party code audit is planned to be entrusted to CertiK (which has completed multiple prior audits for the Operating Entity), with the full audit report to be made public (verification standards consistent with the §8.6 contract audit)	Repository address published; audit report to be published upon completion
Official Channels	Official social accounts (anti-impersonation—only the accounts listed below are official, all others are impersonations): X (Twitter) https://x.com/AELION_TWELVE; YouTube https://youtu.be/pIQGQeiPdDw; Telegram https://t.me/AelionTwelve; Discord https://discord.gg/b5zy5typXN GitHub https://github.com/aelion-twelve	Social accounts published (2026-09-15); primary domain determined

2.3 Naming History and Technology Jurisdiction Layout

Naming history: Aelion Twelve is the name of the project and its operating entity; the name is carried over from the project's internal codename at its founding.

Technology jurisdictional layout: the R&D and patent layout of the core compute-scheduling technology is consistent with the team composition. Chief AI Infrastructure Architect (Tunde Akinwale) and the GPU Infrastructure Lead (Amara Okoye) are based in Nigeria, and the related R&D work is carried out there; patent applications proceed under the standard jurisdictional strategy of a cross-border R&D team, and the application numbers will be published on the official website once assigned (see §2.2).

Naming and symbol availability: the token symbol A12 and the project name Aelion Twelve have undergone trademark and mainstream market-platform uniqueness checks, and no conflict with any existing project or trademark has been found.

2.4 Compliance Commitments

In its operations, the platform complies with: customer identification (KYC) and sanctions-list screening; usage review of compute tenants and content compliance rules (see § 4.7); data protection and cross-border transfer compliance assessments; and, in jurisdictions involving digital assets, Token-related activities are conducted only after obtaining the necessary licenses. The Operating Entity's corporate registration has been completed (Canada); SEC-related license applications are in progress — business involving the aforementioned licenses will not be conducted until the licenses are obtained.

License progress observation point: To make "no business before licensing" externally observable, the Operating Entity maintains a license status table on the disclosure page, listing item by item the application stage of matters such as SEC-related licenses, the disclosure method for acceptance/receipt numbers, the estimated completion time window, and the current status. The first license status table will be published no later than the launch date of the transitional disclosure page (see §2.2) and updated monthly thereafter; the responsible role for updates is the Operating Entity's legal and compliance lead (matters involving external law firms are also annotated in the table with the responsible firm). Any announcement of a licensed business going live must include a query link to the corresponding license status; if the link fails, "regulator name + acceptance/receipt number + the regulator's official query portal" serves as the alternative verification method, and the link must be repaired or replaced within 5 business days.

2.5 Funding and Resource Transparency Arrangements

Public-beta incentives, third-party accounting attestation, contract security audits, compute procurement and node revenue sharing, and team compensation are all funded by the Operating Entity's own capital. In particular, the third-party code audit of the core scheduling and verification modules and the §8.6 settlement contracts is planned to be entrusted to CertiK (which has completed multiple prior audits for the Operating Entity); the full audit report will be published on the official website upon completion. The specific amounts, caps, and funding arrangements for each expenditure item will be disclosed in subsequent versions.

Chapter 3 System Architecture Overview

3.1 Design Principles

The system design follows four principles, each corresponding to verifiable engineering implications:

1. **Physical-topology affinity: the scheduling destination of a task is determined by its communication-compute characteristics, not by its "degree of decentralization."** Training tasks stay in high-bandwidth clusters; inference tasks sink to the edge.
2. **Heterogeneous hardware abstraction: what is unified is the task submission interface and metering basis, not the underlying execution environment; vendor differences are disclosed through an explicit adaptation matrix rather than hidden.**
3. **Layered verifiability: trust comes from the stacking of multiple layers of mechanisms (declaration, benchmarking, telemetry, task verification, reputation); bypassing any single layer does not constitute a systemic risk.**
4. **Vertical value chain: CaaS (compute) → MaaS (model services) → AaaS (agent applications) add value layer by layer, and platform revenue gains unit price and stickiness as it moves up the value chain.**

3.2 Four-Layer Architecture

Layer	Name	Core Responsibilities
Layer 1	Resource Access and Trusted Verification Layer	Node onboarding, resource description, capability benchmarking, status monitoring, reputation recording
Layer 2	Scheduling and Resource Pooling Layer	Task profiling, topology awareness, dual-path scheduling, resource leases, lifecycle management, tenant console and sandbox runtime
Layer 3	Model Services Layer (MaaS)	Model onboarding and version management, vertical model pipelines, API gateway, inference performance engineering, service metering
Layer 4	Agent and Application Layer (AaaS)	Agent orchestration canvas, developer toolchain, prebuilt industry agents, open platform and unified checkout, commercial settlement

Layers are decoupled through standard interfaces. The control plane (scheduling, metering, settlement) is separated from the data plane (model execution, data transfer): data-plane transfers are completed preferentially within the cluster where the task resides, and the control plane does not forward large-scale tensor data.

A12 四层系统架构：纵向价值链逐层增值，控制平面与数据平面分离



Figure 3-1 A12 four-layer system architecture: the vertical value chain adds value layer by layer, with the control plane separated from the data plane

3.3 Ecosystem Roles

Role	Input	Return Received	Risks and Obligations
Compute Provider (Worker)	GPU resources and online time	Compute service revenue (priced in fiat/stablecoins, with additional Token incentives)	Compliance with task lifecycle rules; abuse compensation clauses in §4.7
Validator Node	Verification compute power and security deposit	Verification service compensation	Must stake a security deposit; slashing for misconduct
Model Developer	Models and adaptation capabilities	Share of invocation revenue	Model license compliance
Agent/SaaS Developer	Orchestrated applications and tools	Share of application revenue	Application compliance and content responsibility
Tenant/Enterprise	Real business demand	Model and agent services	Data authorization compliance

3.4 Design Reference Frame (Engineering Lessons from Ethereum)

A12's mechanism design draws heavily on the engineering paradigms of Ethereum—a distributed system validated at massive scale—without copying its consensus mechanism: what Ethereum validates is ledger state, while what A12 validates is computation processes and performance; the difference in the objects of validation determines the difference in mechanism form.

Special note: the table below is a conceptual analogy at the mechanism-design level, intended to help readers understand A12 in the language of mature systems; it does not imply that A12 has the same degree of decentralization, trust model, or security assumptions as Ethereum. A12 is a centrally operated platform (see §2.1) and currently does not issue a public chain and does not constitute a Layer2 or any blockchain network; the self-developed public chain and DEX have been completed but are not elaborated in this document, and their introduction is premised on regulatory assessment (see §8.6).

The mapping is as follows:

Reference System Mechanism	A12 Corresponding Design	Section
Ethereum account model and nonce sequence	Operator account system: reputation, settlement, and penalties aggregated by account; multiple devices cannot isolate liability	4.5
Ethereum Gas metering	Task metering uses the dual basis of "CU · duration + token count," recorded per transaction and exportable for audit	5.3 / 6.5
EIP-1559 base fee + tip price discovery	Scheduling price = published base price (floating with supply and demand under public rules) + quality premium, eliminating covert adjustments	5.3 / 7.5
Ethereum smart contract	The protocol layer deploys settlement attestation contracts on a	8.6

Chapter 4 Layer 1: Resource Access and Trusted Verification

4.1 Design Principles and Boundaries (Important Statement)

In open environments, resource verification offers no "absolute anti-counterfeiting." A12's design goal is to make the comprehensive cost of cheating (probability of detection $\times$ loss) significantly higher than the gain from cheating, and to enable anomalous nodes to be continuously identified and downgraded. To this end, the platform explicitly commits to "three things we will not do":

1. We do not use any "silicon physical fingerprint" as device identity, nor do we impose fingerprint-based permanent bans (physical characteristics drift with temperature, voltage, and aging; they are neither stably reproducible nor able to support permanent penalties).
2. We do not use zero-knowledge proofs to certify analog quantities of the physical world (a proof system can prove that "a piece of computation is correct"; it cannot prove that "a physical event occurred").
3. We do not make access to tenant model weights or data a precondition for verification (protecting tenant privacy).

4.2 Five-Layer Trusted Verification System

Layer	Mechanism	Description
L1 Declaration Verification	Collects GPU model, VRAM, drivers, and PCIe/NUMA topology, cross-checked against a known specification library	Collected via official interfaces such as NVML/ROCm; explicitly runs in user space and can be deeply tampered with, so it serves only as the first filter
L2 Capability Benchmarking	Platform-issued random benchmark challenges: GEMM matrix multiplication, VRAM read/write, tensor-core stress tests, in multiple combinations of scale and precision	Challenge seeds are generated by the platform's validation cluster using a random beacon (VRF) and cannot be predicted by nodes; nodes return result digests, elapsed time, and telemetry
L3 Runtime Telemetry	Continuously collects utilization, VRAM, temperature, power consumption, and task execution metrics	Used to detect "declaration inconsistent with reality" and signs of overselling
L4 Task-Level Verification	Selected by task type: sampled re-execution (platform validation cluster), redundant computation (multi-node comparison), numerical tolerance determination, and interactive replay localization in disputes	Generative outputs use a combination of structured validation and manual spot checks, not numerical similarity as the sole criterion
L5 Historical Reputation	Aggregated by operator (see §4.5): uptime, performance deviation, task success rate, verification pass rate	Reputation affects scheduling priority, task admission, and settlement cycle

A12 五层可信核体验体系：单层被绕过不构成系统性风险

信任 = 声明 × 实测 × 遥测 × 任务验证 × 声誉（分层叠加，逐级增强）



Figure 4-1 A12 five-layer trusted verification system: layers are stacked; bypassing a single layer does not constitute a systemic risk

Layers L2 and L3 of the five-layer verification system use quantifiable statistical determination methods, described below.

Determination model for L2 capability benchmarking. The platform validation cluster generates random benchmark challenges: random matrices $A \in \mathbb{R}^{(m \times k)}$ and $B \in \mathbb{R}^{(k \times n)}$ (elements uniformly distributed, seeds derived from a VRF beacon); the node executes the GEMM computation $C = A \times B$ at the specified precision (FP16/BF16) on the target GPU and returns the result digest (SHA-256(C)) and the actual elapsed time T . Determination uses two sets of statistics:

- **Correctness:** the validation cluster re-checks sampled digests against a reference implementation and requires the relative Frobenius norm error $\|C - C_{\text{ref}}\|_F / \|C_{\text{ref}}\|_F \leq \epsilon$ (ϵ is set per precision tier, typically on the order of 10^{-2} for FP16, covering legitimate deviations arising from floating-point non-associativity and different operator implementations);
- **Performance plausibility:** $T_{\text{hat}} = 2 \cdot m \cdot k \cdot n / \eta_{\text{peak}}$ is the expected elapsed time derived from the measured throughput baseline η_{peak} (m, k, n are the matrix dimensions of the challenge GEMM), and $(1 - \delta) \cdot T_{\text{hat}}$ is its reasonable lower bound. A measured elapsed time below this bound ($T < (1 - \delta) \cdot T_{\text{hat}}$) is judged as an "inflated compute power" or "precomputed cache" attack; significantly slower tasks do not affect challenge passage and are handled by the SLA and reputation mechanisms (see §4.5 and §5.3).
- **Baseline calibration and contamination protection:** η_{peak} is periodically measured and calibrated by the platform validation cluster against official reference hardware tiers; the baseline table is public and versioned. Direct aggregation of historical measurements from non-onboarded nodes is not used as the baseline source, preventing low-quality samples from dragging the baseline down and disabling detection, or high-quality samples from raising the baseline and falsely harming honest nodes.
- **False-positive protection for high-performance nodes:** outperforming the baseline is a legitimate case (capability upgrades, see §4.6); the inflation process is triggered only when measurements fall below the physical lower bound. An operator so judged may apply for remote re-testing and manual review (submitting measurement records and runtime environment descriptions); during the review, the related revenue is frozen in Pending status, and a false positive results in full unfreezing with no negative reputation record.

Multiple matrix sizes (from $1K^2$ to $16K^3$) and precision combinations are randomly selected, so nodes cannot optimize for a single size.

Anomaly detection for L3 runtime telemetry. Exponentially weighted moving averages (EWMA) are used to maintain baselines for time-series metrics such as utilization, VRAM usage, temperature, and power consumption:

$$s_t = \alpha \cdot x_t + (1 - \alpha) \cdot s_{t-1}$$

where α is the smoothing coefficient (typically 0.05–0.2). When an instantaneous value deviates from the baseline by more than 3σ (σ estimated from the EWMA variance of historical residuals) and persists for K sampling windows, it is flagged as a telemetry anomaly event; accumulated anomaly events feed into operator reputation (see §4.5) and trigger increased-density spot checks for the corresponding tasks. The boundary of this method is equally clear: what telemetry detects is "statistical inconsistency," not an exhaustive proof of tampering methods—which is precisely why five stacked layers rather than single-point determination are needed.

Formalization of the challenge protocol. The generation and disclosure of L2 challenges follow the Commit–Reveal paradigm (a paradigm sharing its origin with Ethereum's random-number beacon): each round, the validation cluster first publishes a challenge commitment and later reveals the seed, so that nodes can afterwards independently audit that the validation cluster "did not selectively choose questions":

$(y, \pi) \leftarrow \text{VRF.Eval}(sk_V, epoch)$; $\text{Verify}(vk_V, epoch, y, \pi) = 1$ $Com_q = \text{SHA-256}(seed_q \parallel \epsilon \parallel \delta)$ (the commitment binds only the seed and public parameters) — $seed_q$ is revealed after challenge execution

where (y, π) are the VRF output and proof, and sk_V / vk_V are the validation cluster's signing key pair; any third party can use vk_V to verify the seed's legitimacy offline. Note that the commitment value must contain only $seed_q$ and the public parameters (ϵ, δ) : A_q and B_q are derived from $seed_q$; if they were included in the commitment, the matrix contents would be locked at the commitment stage, and the "commit first, reveal later" design against selective problem selection would fail. The reference determination flow for challenge verification is as follows (illustrative pseudocode, not the final implementation):

```
def verify_challenge(node, seed, params):
    # 1. Derive unpredictable challenge matrices from the VRF
    seed (size and precision determined by the seed)
    A, B = derive_challenge_matrices(seed,
    params.sizes, params.precisions)
    C_ref = reference_gemm(A, B, precision=params.precision) #
    validation cluster reference implementation
    # 2. Correctness: relative Frobenius norm error <= ε
    ok_math = frobenius_rel_err(node.reply.C, C_ref)
    # 3. Performance plausibility:
    measured elapsed time not below (1 - δ) × the physical lower bound
    m, k, n = A.shape[0], A.shape[1], B.shape[1]
    T_hat = 2 * m * k * n / throughput_baseline(node.tier)
    ok_time = node.reply.T >= (1 - params.delta) * T_hat
    if not (ok_math and ok_time):
        telemetry.flag(node.operator, "L2_FAIL",
        magnitude=abs(node.reply.T / T_hat - 1))
    return ok_math and ok_time
```

4.3 Validator Network (Clear Responsibilities and Costs)

- **Validation cluster:** the core validation cluster built and operated by the platform, responsible for all sampled re-execution and for the generation, dispatch, and determination of benchmark challenges; its costs are included in the platform service fee and transparently itemized to tenants.
- **Volunteer validator nodes:** high-reputation nodes may apply to join the validator network and must stake a security deposit (the amount tied to the verification workload they intend to undertake); they are

compensated for completed valid verification. Verification errors or collusion with the verified node result in slashing of the deposit and permanent disqualification from verification.

- **Random beacon:** challenge seeds are generated by the validation cluster using a verifiable random function (VRF), with seed commitments published; nodes cannot predict challenge contents, and the validation cluster can prove after the fact that it did not selectively choose problems.

4.4 Dual-Track Node Access

Mode	Applicable To	Tasks Carried	Staking Requirement	SLA
Elastic Best-Effort Mode	Personal workstations, idle GPUs, edge devices	Interruptible, retryable batch processing, embedding, RAG data processing, offline inference	No upfront staking	Not committed; tasks are automatically retried on failure
Dedicated Guaranteed Mode	Enterprise IDCs, professional clusters, high-availability workstations	Online inference, model services, enterprise APIs	Revenue escrow or security deposit (per agreement)	Committed availability targets, redundant deployment, breach compensation

Elastic nodes carry no SLA business—enterprise-grade availability is guaranteed by redundant deployment in the dedicated resource pool, architecturally eliminating the contradiction between "going offline as soon as the mouse and keyboard become active" and "online services."

4.5 Anti-Sybil and Anti-Overselling Mechanisms

1. **Operator account system:** all reputation, settlement, and penalties are aggregated at the level of the operator (natural person or legal entity). The performance of multiple devices under the same operator is computed jointly; a single device's breach affects the operator's overall reputation and settlement cycle, so "registering many new identities and misbehaving once with each" cannot isolate losses.
2. **Global resource leases and cross-identity VRAM mutual exclusion:** each GPU is registered in the platform ledger under the key "device serial number + operator account"; while a resource is locked by a task, no second sellable resource may be created under the same key. More critically, the device serial number is globally unique at the ledger level—the same serial number may be bound to only one operator account at any given time, with at most one valid lease, regardless of which account declares it; changing the account binding of a serial number requires review by both parties and a 30-day cooling-off period. This makes "one physical card swapped across three identities, each declaring its full VRAM once" mutually exclusive at the ledger level, rather than relying on after-the-fact detection. Repeated declaration of the same physical device (including multi-identity declarations after virtualization partitioning) triggers account-level review; the hardware combination collected by L1 declaration verification (GPU model, total VRAM, PCIe/NUMA topology, VBIOS version, and serial number) is additionally used for cross-account deduplication—as a deduplication and review signal only, not as device identity proof or grounds for permanent bans (consistent with the "three things we will not do" in §4.1). To make this mutual exclusion externally observable, cross-identity deduplication statistics (declared device count / deduplicated device count / difference) are included in the "node scale" field of each phase's acceptance report and published together (see §9.2.1).

3. **Topology-clustering suspicion detection:** Cluster analysis is performed on the PCIe topology, NUMA structure, and driver/firmware combinations of declared resources. When "multiple identities suspected of belonging to the same physical machine" are detected, the accounts are rate-limited for observation and manual review is triggered. Topology features are used solely for cluster detection and scheduling reference, not as proof of device identity. Misjudgment protection and appeal channel: Compliant multi-machine operators (Internet cafes, campus computer labs, small studios, reseller IDCs) inherently share identical hardware and firmware combinations; they may submit device inventories, purchase receipts, and other materials to apply for operator-identity review (processing time: 7 business days, with results published). The reputation model distinguishes "confirmed cheating" (slashing) from "cluster suspicion" (rate-limited observation only, with no slashing, no permanent demotion, and immediate restoration of weight upon passing review).
4. **Earnings growth curve for new nodes:** Newly registered operators are subject to an earnings cap with progressive release during their first 30 days, driving the expected one-time profit of bulk-registered accounts toward zero; the cap is gradually raised as an honest track record accumulates.
5. **Deferred earnings release pool:** Node income flows through three states: Earned -> Pending -> Released. During the Pending observation window (tiered at 1-72 hours according to task value), earnings are subject to spot checks and dispute handling; if a violation is detected, the corresponding Pending earnings are slashed and used to compensate affected tenants. A separate security deposit applies to high-value SLA workloads; see §4.4.

Quantitative reputation model (design target). An operator's reputation score R in $[0, 100]$ is updated via "exponential decay + task feedback":

$$R_{t+1} = \lambda \cdot R_t + (1 - \lambda) \cdot r_{\text{task}}$$

where r_{task} in $[0, 100]$ is the per-task rating (combining verification pass rate, performance deviation, and SLA attainment), and λ in $(0, 1)$ is the decay factor — the closer λ is to 1, the greater the weight of history, the more incidental fluctuations are smoothed, but the slower the recovery; the closer λ is to 0, the greater the influence of new ratings, the more sensitive the score is to recent changes, and the faster the recovery. This design balances "stability" against "responsiveness": a long-standing good record smooths incidental fluctuations, while sustained degradation causes R to converge monotonically to the level of recent performance. Violations incur an additional penalty multiplier: each confirmed violation event multiplies R directly by a decay coefficient κ (set to 0.8, adjustable by risk-control rules) and is recorded in the operator-level review process. The mapping between reputation R and scheduling weight / admission thresholds (e.g., $R < 60$ bars entry into the SLA resource pool) is published as a design target after calibration during Phase 0 internal testing.

The slashing function adopts the industry-standard collateral/slashing model; the slashing cap strictly does not exceed the operator's custodial deposit, avoiding "negative-balance" accounts:

$$\text{Slash}(o) = \min(m_{\text{violation}} \times V_{\text{disputed}}, D_{\text{escrow}}(o)); \quad R_o \leftarrow \kappa \cdot R_o$$

where V_{disputed} is the value of the disputed task, $m_{\text{violation}}$ is the liability multiplier (initial value 1.0, escalating for repeat violations), and $D_{\text{escrow}}(o)$ is the operator's escrow deposit — the slashed amount is hard-capped by the deposit; any excess is covered by the platform's abuse compensation fund, and the operator bears no unlimited liability.

(For a contract-level illustration of earnings escrow and slashing, see the protocol layer in §8.6 — this capability is a Phase 4 deliverable and is presented uniformly in the protocol-layer chapter to avoid being misread as already delivered.)

4.6 Dynamic Capability Rating (CU)

A node's composite capability score is defined as:

$$CU(i) = w_P \cdot P(i) + w_B \cdot B(i) + w_M \cdot M(i) + w_N \cdot N(i) + w_R \cdot R(i)$$

where P is the compute benchmark score, B is the VRAM bandwidth score, M is the VRAM capacity score, N is the network quality score, and R is the historical reliability score; the weights w_P , w_B , w_M , w_N , and w_R are adjusted by task type (training tasks raise the P/N weights; online APIs raise the N/R weights), subject to $w_P + w_B + w_M + w_N + w_R = 1$. All sub-scores derive from L2 benchmarking and L3 telemetry rather than vendor nominal values.

To ensure comparability across heterogeneous hardware, each sub-score is first min-max normalized to [0, 1]:

$$X'(i) = (X(i) - X_{min}) / (X_{max} - X_{min})$$

where X_{min} / X_{max} are taken from the measured-distribution quantiles (P5 / P95) of the same hardware generation tier, suppressing outlier stretching. The normalized scores are weighted-summed per the formula above to yield CU(i) in [0, 100], which is then divided into Tier-S/A/B/C according to calibrated cut points. Cut-point selection follows the principle of "minimizing intra-tier CU dispersion while maximizing cross-tier differences in task pass rates," and is published after regression calibration against actual task pass rates during Phase 0 internal testing (before calibration, the hardware specification table serves as the interim basis for tiering).

Nodes are divided into four tiers — Tier-S/A/B/C — based on measured CU. The correspondence between tiers and permissible task scopes is given in the scheduling constraint table in §5.4, and the task table for each tier remains verifiably consistent with hardware specifications.

4.7 Node Owner (Supplier) Protection and Content Compliance

The platform's obligations to node owners are reciprocal to those of tenants:

- **Least-privilege execution:** Tenant tasks run in container-isolated environments (cgroups v2 + Seccomp + read-only root filesystem), and the Node Daemon runs as an unprivileged user. Isolation significantly shrinks the attack surface but cannot absolutely eliminate risk; the platform therefore carries insurance and maintains an abuse compensation fund.
- **Usage review:** Tenant registration requires KYC; task images are distributed through whitelisted registries, and nodes are prohibited from pulling arbitrary external images; uses such as cryptocurrency mining, port scanning, and spam traffic are placed on a prohibition list and detected via telemetry.
- **Abuse compensation:** Losses suffered by node owners due to lapses in the platform's risk control are compensated from the abuse compensation fund in accordance with the agreement.
- **Freedom of exit:** A node may exit at any time; exit does not affect the withdrawal of earnings already in the Released state.
- **Confidential computing outlook (research item):** For "privacy protection of tenant code and weights in node memory," we track the confidential computing route based on Trusted Execution Environments (TEEs) — namely, data-center GPUs supporting CC (Confidential Computing) mode, which can encrypt VRAM and the computation process at the hardware level so that even the node owner cannot read tenant data. This capability depends on specific hardware generations and vendor support; it is placed on the research horizon (see §9.3) and is not a committed item until a sufficient scale of compliant devices exists

within the ecosystem. Until then, container isolation + usage review + the compensation fund remain the primary protection combination.

Chapter 5 Layer 2: Scheduling and Resource Pooling

5.1 Hardware Abstraction Layer (HAL) — An Honest Heterogeneity Strategy

A12 unifies the task submission interface through a hardware abstraction layer rather than claiming to eliminate underlying differences. To state it explicitly:

- **Unified interface:** Tenants submit tasks using a unified task description (model, data, compute requirements, latency and budget constraints) without selecting a specific GPU model.
- **Heterogeneous backends:** Underlying execution depends on each vendor's drivers, compilation toolchains, and operator implementations. The roadmap commits to production readiness for the NVIDIA CUDA stack; the AMD ROCm stack is delivered as experimental support; adaptation for domestic chips is listed as a research item and will be announced separately according to actual engineering progress.
- The platform does not use any product name similar to third-party trademarks.

5.2 Dual-Path Scheduling

The scheduler selects a path based on the task's communication-to-computation ratio α_{comm} :

$$\alpha_{comm} = T_{comm} / T_{compute} = (\text{cross-node communication data volume} / \text{effective link bandwidth}) \div (\text{computation volume} / \text{actual GPU throughput})$$

- $\alpha_{comm} \geq 1$ (communication-bound): Scheduled to centralized compute centers (cloud-platform GPU clusters, professional AI IDCs, HPC) — A12 integrates these resources through standard adapters and provides unified orchestration, replacing tenants' own price comparison and stitching.
- $\alpha_{comm} < 1$ (compute-bound): Scheduled to the A12 distributed resource pool (verified IDCs, workstations, and edge nodes), executing locally closed-loop tasks (inference, Embedding, LoRA/QLoRA, batch processing) and avoiding high-frequency wide-area synchronization.

The two time components of α_{comm} are modeled using the classic Hockney communication model and Roofline throughput estimation:

- **Communication time:** $T_{comm} \approx T_{startup} + m / BW + o(m)$, where m is the data volume to be transferred per synchronization step (in full-parameter training, approximately the bytes of parameters to be synchronized $\times$ the number of synchronizations), BW is the effective link bandwidth (measured, not nominal), $T_{startup}$ is the fixed startup latency (typically 10-100 ms over WANs, typically $< 10 \mu s$ over NVLink/IB), and $o(m)$ is the protocol-stack overhead term;
- **Computation time:** $T_{compute} = \text{FLOPs} / \eta_{eff}$, where FLOPs is the floating-point operations per step (for full-parameter training, approximately $6 \times \text{parameter count} \times \text{token count}$), and η_{eff} is the measured effective compute throughput (bounded by arithmetic intensity under the roofline model, typically 30%-60% of peak compute).

Take full-parameter fine-tuning of a 7B-parameter model as an example: under data parallelism, each step requires synchronizing about 28 GB (FP16 weights + optimizer states, varying by implementation); over a 25 Gbps WAN link

(effective bandwidth ≈ 3 GB/s), single-step communication takes ≈ 9.3 seconds, while the same synchronization volume takes < 1 second within an InfiniBand 400 Gbps (effective ≈ 45 GB/s) cluster — communication time differs by an order of magnitude, whereas single-step computation at this scale takes only a few seconds. This is precisely the quantitative basis for "high-communication tasks must remain in centralized clusters." It should be emphasized that 25 Gbps is on the order of dedicated lines / data-center interconnects; recalculating with a residential 1 Gbps uplink (effective ≈ 110 MB/s), the same 28 GB synchronization volume would take about 265 seconds per step — nearly $30 \times$ the example above — so under real edge bandwidth distributions the conclusion is far more extreme than the example suggests. The scheduler's α_{comm} parameter library is built from the bandwidth distribution of target supply endpoints (uplink P50/P90, cross-border latency) and retains tolerance for estimation error: boundary tasks on either side of the threshold are assigned based on measured re-verification. The scheduler estimates α_{comm} at task submission and persists it, regressing it against observed values from actual task execution to continuously refine the bandwidth and throughput parameter library. The estimated routing of five typical task categories is shown in Figure 5-1:

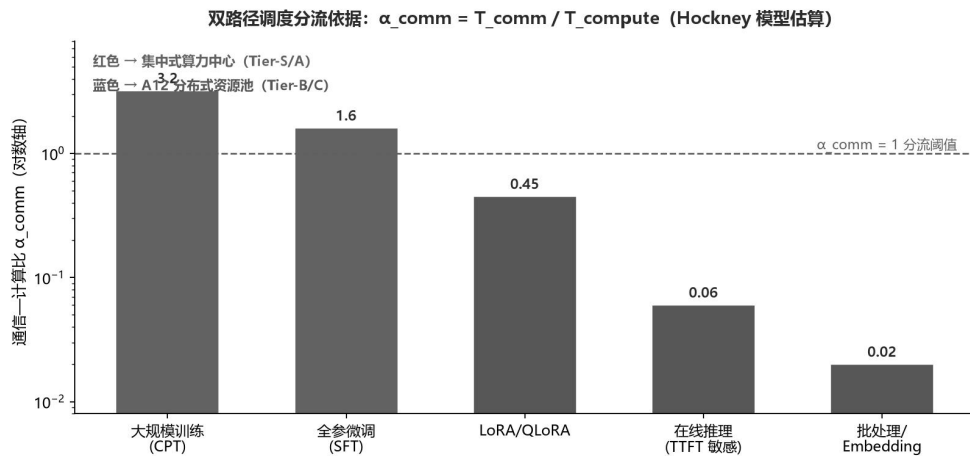


Figure 5-1 Basis for dual-path scheduling: centralized clusters and the distributed resource pool lie on opposite sides of the α_{comm} threshold

Figure 5-1 Data Recalculation Notes: the routing conclusions for the five task types in this figure are supported by a unified parameter table, provided in Appendix C Table D-1 with this edition, including the communication-to-computation ratio α_{comm} estimation range for each task (listed by bandwidth tier), T_{comm} and T_{compute} values with hardware-tier assumptions, the routing threshold, and measured points for boundary tasks; readers can independently recalculate tier by tier using $\alpha_{\text{comm}} = T_{\text{comm}} \div T_{\text{compute}}$. The α_{comm} ranges are design estimates derived from the §5.2 Hockney communication model and Roofline throughput estimation; they are reviewed and updated quarterly as Phase 0 measurement samples expand, with update records archived on the disclosure page.

5.3 Scheduling Decisions and the SLA Model

The scheduling score is a multi-objective weighted sum:

$$\text{Score}(n, t) = w1 \cdot P_{\text{compute}} + w2 \cdot P_{\text{memory}} + w3 \cdot P_{\text{network}} + w4 \cdot P_{\text{reliability}} + w5 \cdot P_{\text{SLA}} - w6 \cdot C_{\text{compute}} - w7 \cdot C_{\text{network}}$$

Weights are dynamically adjusted by task type. The scheduling problem is engineering-simplified into two steps — constraint filtering + greedy scoring: first filter the candidate set by hard constraints (tier admission, SLA admission, VRAM/topology match, reputation threshold), then rank the remaining candidates by Score and select the best. The overall complexity is $O(N \log N)$, and a single decision completes in milliseconds at the scale of thousands of nodes; long-term optimization of the multi-objective weights is listed as a research item and is not a first-version commitment.

The redundancy mathematics of SLA availability. Let a single node's monthly availability be A_0 ; with an active-standby pair deployed independently, with independent power supplies and network egress, the overall service availability is:

$$A = 1 - (1 - A_0)^2$$

Taking $A_0 = 99.5\%$ as an example, after dual-machine redundancy $A = 1 - 0.005^2 = 99.9975\%$, i.e., cumulative monthly unavailability shrinks from about 3.6 hours to about 1.1 minutes. The platform performs capacity planning accordingly: the target availability A^* is inverted to obtain the required single-node availability $A_0^* = 1 - \sqrt{1 - A^*}$, and new orders are accepted only when the number of active-standby pairs in the dedicated resource pool satisfying $A_0 \geq A_0^*$ is at least the number of sold SLA orders plus a safety margin — when the margin is insufficient, new SLA orders are rejected rather than oversold; this is the capacity-level implementation of the dual-track isolation in §4.4. It must be emphasized that both 99.5% and 99.9975% above are example calculations under the design-baseline definition and are likewise design targets rather than measured conclusions; the example illustrates the order-of-magnitude relationship of redundancy composition, and the 99.9975% figure is not an externally committed value (for the externally committed floor, see the SLA Target Value Framework below).

Control-plane availability (A_{platform}). The formula above covers only the resource-node side; control-plane components such as scheduling, metering, settlement, and the validation cluster are built and operated by the platform, and their availability is modeled separately: the control plane is deployed across multiple availability zones with multi-active WSS gateways; if the validation cluster is unavailable, challenge issuance automatically degrades to sampling mode, and Pending reward release is deferred without counting as node default. The externally committed service-level availability = $A_{\text{platform}} \times A_{\text{node}}$; its design target is given in the "SLA Target Framework" below, and the formal committed value will be published on the disclosure page before the public beta launches. **Billing rules during control-plane failures:** billing is suspended during the failure period, and orders that have met the SLA are handled per the compensation clause—tenants do not pay for platform failures.

SLA Target Value Framework (a precondition for external sales in Phase 2). The externally committed service-level availability targets will be published together with the official release; this document first provides the design targets: service-level availability under the dedicated guarantee mode $A_{\text{service}} \geq 99.5\%$, composed of control-plane availability $A_{\text{platform}} \geq 99.9\%$ and node-side composite availability $\geq 99.6\%$ (composed as $A = A_{\text{platform}} \times A_{\text{node}}$). The three definitions are clarified as follows. First, 99.5% is the externally committed floor, not the expected operating value — capacity planning follows the logic above of inverting A^* to obtain A_0^* ; it does not conflict with the earlier 99.9975% example, which is the theoretical composite value under the design baseline $A_0 = 99.5\%$ (monthly basis), and the difference between the floor and the theoretical composite value is the safety margin

of the external commitment. Second, the design baseline of A_0 is 99.5% per month (active-standby nodes under the dedicated guarantee mode); inverting $A = 1 - (1 - A_0)^2$, the externally committed node-side composite availability $\geq 99.6\%$ corresponds to $A_0 \geq 93.68\%$ — this 93.68% is the inverted value corresponding to the externally committed floor, not the single-node design baseline, and actual capacity planning follows the 99.5% baseline. Third, the statistical window: A_{service} , A_{platform} , and A_{node} are all measured per calendar month (monthly window); annual availability is the arithmetic mean of the 12 calendar months, and SLA compensation is settled on a calendar-month basis. All of the above figures are design targets rather than measured conclusions; before the target values are formally published, the platform will not sign any external service contract containing an SLA — SLA commercialization in Phase 2 is strictly preconditioned on that publication; the "no billing during outages" billing and compensation rules for platform failures will be published together with the target values.

Supply-side economics and disposal of dedicated-pool surplus. The comparative calculation of the monthly revenue range, electricity and depreciation shares, and the opportunity cost of "idling / reselling" for a single RTX 4090-class device under the elastic model will be published with the node-recruitment documentation — this calculation is the first-principles basis for the node-retention and scale targets in § 9.2. When qualified active/standby pairs in the dedicated pool persistently exceed sold orders, idle capacity is diverted to the elastic pool to take interruptible tasks, or is settled with enterprise tenants under capacity-reservation agreements, avoiding one-sided idleness.

Supply-side economics calculation framework (structure first, figures later): The calculation table is published in a unified field structure, with units and value definitions for each field agreed as follows — ① Device model and tier: listed by measured CU tier (e.g., RTX 4090 class, 24 GB VRAM), with the number of sampled nodes annotated; ② Electricity price range: RMB yuan per kWh (tax included), tiered by the supplier's country and residential/commercial tariff, with cross-border samples additionally annotated with currency and exchange-rate basis; ③ Hardware depreciation basis: straight-line method, 36-month depreciation period, 5% residual value rate (published together with the device tier, with reasons explained when changed); ④ Monthly sustainable task volume: CU·hours per month, taken as the measured P50 of same-tier nodes, with P90 listed for comparison; ⑤ Task unit price range: USD per CU-hour, using the published base-price definition in § 5.3, excluding quality premiums; ⑥ Node monthly income range: USD per month, listing both pre-commission and post-commission figures; ⑦ Opportunity-cost comparison: 0 for the idling case; for the reselling case, the monthly equivalent income (USD/month) converted from the average secondary-market price is listed. The figures will be published together with the official release. The node retention and scale criteria for Phase 1 in § 9.2 take the published version of this calculation table as the effective basis for re-verification; before the table is published, the relevant criteria remain design targets only.

The reference implementation of the scheduling decision is as follows (constraint filtering + greedy scoring, illustrative pseudocode):

```
def schedule(task, pool):
    # Hard-constraint filtering (tier admission / SLA mode / VRAM / reputation
    threshold), complexity O(N)
    cand = [n for n in pool if n.tier in TIERS_FOR(task.kind)
    and (not task.sla or n.mode == "DEDICATED") and n.free_vram >= task.vram
    and n.reputation >= R_MIN(task.sla)]
    if task.sla and sla_headroom(pool) < SLA_MARGIN:
        return Rejected("Insufficient SLA headroom: reject rather than oversell (see §4.4)")
    # Greedy scoring to
    select the best, complexity O(|cand| · log |cand|)
    return max(cand, key=lambda n: score(n, task),
    default=Queued(task))
```

Scheduling pricing mechanism: Task pricing adopts a "published base price + quality premium" structure — the base price floats with supply and demand under public rules (referencing the price-discovery approach of

EIP-1559; see §3.4), and the quality premium is charged according to resource tier (Tier-S/A/B/C) and SLA level. All price parameters are published and queryable; covert adjustment is precluded.

5.4 Resource-Tier Scheduling Constraint Table

The correspondence between tiers and permissible task scopes is as follows (consistent with the CU grading in §4.6):

Tier	Typical hardware (verified by measurement)	Permissible tasks	SLA support
Tier-S	High-end data-center GPUs + high-speed interconnect (NVLink/IB)	Distributed training, CPT, large-scale full-parameter fine-tuning	Dedicated Guaranteed Mode
Tier-A	Data-center-class single machine (A100/H100 class)	Large-parameter online inference, long-context serving, multi-task SFT	Dedicated Guaranteed Mode
Tier-B	Professional workstations (RTX x090 class and above)	General online inference, LoRA/QLoRA, Embedding	Elastic best-effort by default; may apply for dedicated mode upon meeting standards
Tier-C	Consumer-grade GPUs, edge devices	Batch processing, RAG data processing, offline inference, lightweight inference	Elastic best-effort mode only

Note: Tiers are determined by L2-measured CU scores rather than model-based admission; a physical node that fails to meet the standard in measurement is demoted, and one that exceeds it may be promoted; "capability upgrade" is mutually exclusive with the "misreporting determination" in §4.2 — misreporting is triggered only when measured performance falls below the physical lower bound $(1-\delta)\cdot T_{\text{hat}}$, while above-baseline performance is normally reflected through CU tiering (see §4.2 for the anti-misjudgment mechanism).

5.5 Task Lifecycle and Dispute Handling

Task state machine: Pending -> Scheduled -> Running -> (Suspended) -> Completed / Failed / Disputed. For a disputed task, the corresponding Pending earnings are frozen and the task enters interactive replay localization: task slices are replayed on the validation cluster to identify the responsible party (node, platform, or tenant input), and compensation is allocated according to liability.

A12 任务生命周期状态机：争议任务冻结 Pending 收益，进入交互式重放定位责任方

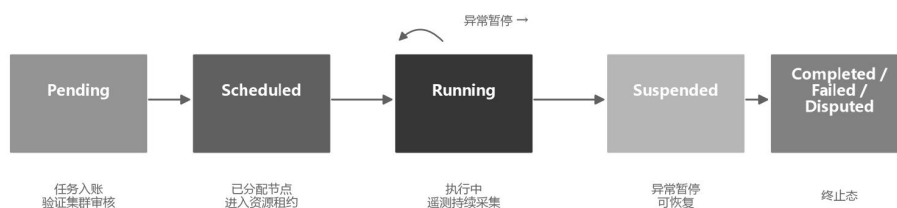


Figure 5-2 A12 task lifecycle state machine: a disputed task freezes its Pending earnings and enters interactive replay to identify the responsible party

5.6 Tenant-Oriented Console and Secure Sandbox Runtime

To deliver a tenant experience no worse than that of traditional cloud platforms, Layer 2 encapsulates a tenant interaction layer and a secure execution environment. All of the following are design goals; acceptance metrics will be published with the Phase 0 closed beta:

1. **Web terminal and file channel:** after registration, a node proactively establishes an outbound encrypted WebSocket (WSS) connection to the scheduling gateway, forming a reverse-proxy tunnel; the tenant console connects directly to the node via a web-based terminal emulator component, eliminating public-IP port mapping and key distribution, and remaining compatible with any NAT/firewall environment. The console also proxies the online Notebook service and the file storage service; uploads use HTTP chunked transfer with resumable upload, ensuring reliable transmission of large datasets and training scripts.
2. **Containerized sub-second instance scheduling:** instead of the traditional virtual-machine model, a lightweight container runtime is used uniformly as the sandbox. The platform pre-provisions a streamlined base image (Ubuntu LTS + CUDA + PyTorch inference stack + common dependencies) and preloads the read-only image layers during idle periods; at task launch, only a few megabytes of custom layers are pulled in real time. Design goal: single-instance cold-start time ≤ 10 seconds (same region, image-cache-hit tier; cross-region or cache-miss scenarios are assigned separate targets tiered by node uplink bandwidth, with measured data subject to the published acceptance report).
3. **Physical-level quota isolation:** cgroups v2 and Seccomp enforce hard limits on container CPU, memory, and IO bandwidth; the NVIDIA Container Runtime ensures that only the allocated GPU devices are visible inside a container. This isolation substantially shrinks the attack surface and prevents side processes from reading tenant memory, but it does not promise absolute prevention of container escape (residual risks are discussed in Chapter 10); the platform backstops this with the abuse compensation fund.

Chapter 6 Layer 3: Model Services (MaaS)

6.1 Model Onboarding and Version Management

A unified Model Registry records model provenance, version, weight digests, runtime environment, and evaluation results; enterprise private models are hosted only, with no change to intellectual-property ownership. This commitment is bounded by the §4.7 security mechanisms and the §5.6 isolation measures: under the elastic-node model, IP protection relies on container isolation (absolute prevention of container escape is not promised; residual risk is backstopped by §4.7 insurance and the abuse compensation fund); tenants with strong IP-isolation requirements are advised to use the dedicated-assurance mode, or the TEE confidential-computing tier outlined as forward-looking in §4.7 (research item). Listing a model requires submitting a Model Card that records intended use, known limitations, and evaluation benchmarks, linked to the platform metering system.

6.2 Vertical-Industry Model Pipeline

The platform plans to build model-layer competitiveness by "breaking through one vertical at a time"; the first vertical is Web3 code and security engineering (consistent with the already-disclosed team background; all metrics in this section are planning targets, not accomplished facts):

1. **Corpus governance:** a structured data-collection pipeline is built to collect and license-screen open-source contract repositories, public vulnerability-disclosure communities, and verified on-chain contract source code; after syntax-tree filtering, near-deduplication, and manual sampling, the pre-training corpus is formed. Scale target: the first batch of pre-training corpus is ≥ 10 B tokens. License governance: a license whitelist/blacklist is established—permissive licenses (MIT/Apache/BSD) are prioritized; the inclusion

boundary for copyleft-licensed corpus such as GPL/AGPL, and the legal position on whether "training weights constitute a derivative work," are subject to a dedicated legal opinion, which will be published together with the corpus-pipeline documentation upon completion; the deduplication and license-screening rules are published with the corpus-pipeline documentation. 10B tokens is the first-batch scale; model size is planned in step with corpus volume per the Chinchilla scaling law (efficiency-oriented small-to-medium parameter tiers); the data budget and timeline for subsequent batches will be published with the model R&D roadmap.

2. **Continual pretraining (CPT) and supervised fine-tuning (SFT):** starting from an open-source code foundation model, perform domain continual pretraining, then apply supervised fine-tuning with a multi-step reasoning instruction set of "defect localization → attack-path construction → consequence prediction → patch recommendation → regression testing". Training is executed in the centralized compute center (Tier-S echelon), following the dual-path rules of Chapter 5; training scale (instance count, parallelism strategy, context length) is disclosed with each phase budget, with no fixed commitment.
3. **Tool-use alignment (DPO):** model outputs are connected to an external tool sandbox (compile-and-test frameworks, static analysis tools), and preference pairs constructed from tool execution results are used for direct preference optimization, reducing "ghost vulnerability" false positives. This mechanism is a deterministic improvement: model conclusions must be verifiable by tools.

The objective functions of the two training stages are as follows (design-target basis).

The SFT stage uses a masked cross-entropy loss: the loss is computed only over the assistant-reply portion, while the prompt portion is masked out:

$$L_{\text{SFT}}(\theta) = - (1/|y|) \cdot \sum_{t=1}^{|y|} \log p_{\theta}(y_t | x, y_{<t})$$

where x is the input (including the multi-step reasoning instruction template), y is the target output sequence, $|y|$ is the target sequence length, and p_{θ} is the model conditional probability under parameters θ . The multi-step reasoning instruction set (defect localization → attack-path construction → consequence prediction → patch recommendation → regression testing) is organized as structured templates, concentrating the loss on the generation quality of the reasoning chain and its conclusions.

In the DPO stage, preference pairs (y_w, y_l) are constructed from tool execution results — y_w is an output that passes tool verification (compiles successfully, tests pass, static analysis confirms), and y_l is an output rejected by the tools — and optimization proceeds under the constraint of the reference model p_{ref} :

$$L_{\text{DPO}}(\theta) = - E_{\{(x, y_w, y_l)\}} [\log \sigma(\beta \cdot \log(p_{\theta}(y_w|x)/p_{\text{ref}}(y_w|x)) - \beta \cdot \log(p_{\theta}(y_l|x)/p_{\text{ref}}(y_l|x)))]$$

where σ is the sigmoid function and β is the strength coefficient for deviation from the reference model (typically 0.1), implicitly replacing the training of a separate reward model. The engineering significance of this approach is that the preference signal does not depend on subjective human annotation but is anchored to objective tool execution results, making "reducing ghost-vulnerability false positives" a reproducible, regression-testable optimization objective rather than a one-off tuning outcome.

Training-scale planning: the Chinchilla scaling law. The training budget for vertical models is jointly determined by the target loss L , the model parameter count N , and the number of training tokens D , following the Chinchilla-style empirical form:

$$L(N, D) = E + A / N^{\alpha} + B / D^{\beta}$$

where E is the irreducible error, and A, B, α, β are task-dependent empirical coefficients. This formula shows that simply scaling up the model without adding data, or simply adding data without scaling up the model, both enter a region of diminishing marginal returns; accordingly, the platform performs triadic planning of "model scale $\leftrightarrow$ corpus scale $\leftrightarrow$ compute budget", and discloses training scale (instance count, parallelism strategy, context length) with each phase budget, with no fixed commitment.

Compliance boundary: model capabilities are positioned as an auxiliary reference tool, with outputs used for development assistance and internal security assessment; the platform does not issue formal audit reports with CVSS ratings externally — such services must be provided by qualified partners. The engagement process has started, with the goal of completing the signing of at least one qualified partner and disclosing its name before public-beta launch; before signing, this product line will not be formally commercialized externally, and partnership progress will be updated on the official website by phase (subject to the forward-looking statements in Chapter 10).

6.3 OpenAI-Compatible API Gateway

The gateway strictly follows the OpenAI interface standard, enabling zero-code migration of existing applications:

- **Standard endpoints:** backward-compatible `/v1/chat/completions`, `/v1/models`, and `/v1/embeddings` are provided, with server-sent events (SSE) streaming responses; applications already integrated with the OpenAI SDK, LangChain, or LlamaIndex only need to replace the gateway address and API key.
- **Dynamic load balancing:** the gateway distributes traffic according to the inference queue depth of each resource echelon — long-context, high-load tasks are routed to Tier-A and above resources, while high-frequency short tasks are routed to the edge resource pool. Design goal: median time to first token (TTFT) during public beta $\leq 400\text{ms}$ ($\leq 8\text{K}$ context tier; the 32K long-context tier will be assigned a separate target based on public-beta measurements).
- **Load-balancing algorithm:** within the same echelon, the "Power of Two Choices" strategy is adopted — two candidate instances are sampled at random, and the request is dispatched to the one with the smaller queue depth. This strategy requires no global-minimum search, has $O(1)$ communication overhead, yet compresses queue length from the exponential distribution of global random assignment to polynomial decay: the classical queueing-theory result is that, as system load approaches saturation, the power of two choices reduces the average queue length from $O(n)$ to $O(\log \log n)$. Compared with the global-scan "always pick the emptiest instance" scheme, at thousand-instance scale it trades a negligible long-tail cost for the scheduler's own low latency and horizontal scalability, matching the gateway's TTFT target.

Queueing-theory capacity planning. When request arrivals approximate a Poisson process, the per-instance service rate is μ , and the number of concurrent instances is c , the queueing wait time is given by the Erlang C formula (M/M/c model):

$$TTFT \approx T_{\text{prefill}}(s) + W_q, \quad W_q = C(c, \rho) / (c\mu - \lambda), \quad \rho = \lambda / (c\mu)$$

where $T_{\text{prefill}}(s)$ is the compute time of the prefill stage (growing linearly with context length s), and $C(c, \rho)$ is the Erlang C waiting probability. This model informs two engineering decisions: first, given the TTFT target (400ms), back-solve for the minimum number of concurrent instances c required per echelon; second, trigger scale-out before queue depth approaches saturation ($\rho \rightarrow 1$) — which is precisely the theoretical basis for the load balancer using

queue depth as its routing signal. During public beta, after calibrating μ against the measured arrival rate λ , the capacity-planning table will be published.

6.4 Inference Performance Engineering: Continuous Batching and Cold-Start Optimization

- **Iteration-level continuous batching:** requests are dynamically inserted and evicted at iteration granularity; as soon as a request completes, its VRAM pages are reclaimed immediately, and new requests enter the ready queue awaiting insertion at the next iteration, eliminating the "bucket effect" of static batching.
Design goal: GPU effective utilization $\geq 75\%$ (subject to measurements in the published acceptance report).
- **Paged VRAM management:** a paged key-value cache (a PagedAttention-style scheme) splits the KV cache into fixed-length VRAM pages and maintains the logical mapping via a page table, supporting non-contiguous physical allocation and reducing fragmentation; it also supports prefix VRAM-block sharing across multi-turn dialogue branches.
- **Cold-start caching:** the read-only weights of base models are preloaded on nodes in each resource pool; user-defined LoRA weights (typically only tens of MB) are fetched from the nearest node via a distributed cache and hot-swapped, enabling seamless switching among multiple tenants and multiple models on the same GPU.

KV-cache capacity planning (design target). The capacity requirement under paged management is estimated as follows:

$$M_{KV} = 2 \times L \times H_{kv} \times d_{head} \times s \times b \times \text{bytes_per_elem}$$

where 2 accounts for the two K/V caches, L is the number of layers, H_{kv} is the number of key-value heads (with GQA grouped-query attention, $H_{kv} \ll$ the number of attention heads, significantly compressing the cache), d_{head} is the per-head dimension, s is the sequence length, b is the concurrent batch size, and bytes_per_elem is the number of bytes per element at the chosen precision. For a model with 80 layers, $H_{kv} = 8$, $d_{head} = 128$, FP16 serving a 32K context: the per-sequence KV cache $\approx 2 \times 80 \times 8 \times 128 \times 32768 \times 2 \text{ B} \approx 10.7 \text{ GB}$ — this magnitude shows that the KV cache, rather than the model weights themselves, is often the VRAM bottleneck for long-context serving. The paging mechanism caps the fragmentation upper bound at "less than one page per sequence" (typical page size 16 tokens; theoretical waste $\leq \text{page size} \times \text{number of sequences}$), and prefix sharing lets multi-turn dialogues and requests with identical system prompts reuse the same batch of physical pages.

Throughput model of continuous batching. GPU effective utilization is defined as the fraction of time target operators are actually executing:

$$\text{Util} = T_{\text{busy}} / (T_{\text{busy}} + T_{\text{idle}} + T_{\text{overhead}})$$

In static batching, T_{idle} comes from "fast requests waiting for the slowest request in the batch" (the bucket effect); the larger the batch, the heavier the waste. Continuous batching re-forms batches at iteration granularity, driving T_{idle} toward zero and leaving only the scheduling overhead itself (T_{overhead} , at millisecond scale). Combined with on-demand KV-cache allocation, the scheduler can dynamically maximize the number of concurrent sequences within the VRAM envelope formed by "batch-size cap $\times$ sequence-length cap" — this is the mechanistic basis of the 75% effective-utilization design target (subject to measurements in the published acceptance report), not an arbitrary figure.

Roofline performance model. The peak inference throughput of a single GPU is bounded by arithmetic intensity I (FLOPs/byte):

$$T_{\text{eff}}(I) = \min(\pi_{\text{peak}}, \beta_{\text{mem}} \cdot I)$$

where π_{peak} is the peak compute and β_{mem} is the VRAM bandwidth. For low-arithmetic-intensity workloads (e.g., the decode stage, which is mostly memory-bound), actual throughput is determined by $\beta_{\text{mem}} \cdot l$ rather than peak compute — this is precisely the theoretical basis for techniques such as KV-cache compression, low-precision inference, PagedAttention, and continuous batching, all of which pursue "reducing per-token VRAM access". Performance-engineering outlook (research items, see § 9.3). Frontier directions in the inference-optimization community will be tracked as the continuous evolution roadmap of the MaaS layer, including:

1. **Speculative Decoding:** a generation scheme in which a small model drafts and a large model verifies in parallel, compressing decoding latency severalfold with unchanged output quality; its draft-and-verify structure is a natural fit for A12's validation cluster architecture;
2. **Low-precision inference (FP8 / FP4):** carrying the KV cache and matrix multiplications in hardware-native low-precision formats, proportionally compressing VRAM footprint and bandwidth demand — per the M_{KV} formula in §6.4, FP8 conversion directly doubles the number of concurrent long-context sequences a single GPU can serve;
3. **Prefill/Decode disaggregated serving:** splitting first-token generation (compute-intensive) from subsequent decoding (bandwidth-intensive) across different resource pools, isomorphic to A12's scheduling philosophy of "routing by communication-to-compute ratio";
4. **Tiered KV-cache offloading:** a three-level cache hierarchy of HBM → DDR → NVMe, using temperature-aware page migration to extend the servable context length;
5. **Semantic Caching:** directly reusing cached answers for semantically duplicate requests, significantly reducing compute consumption on knowledge-base Q&A workloads;
6. **MoE expert parallelism:** distributing expert weights of mixture-of-experts models across nodes with dynamic routing, making it possible to "serve hundred-billion-parameter-class models on a distributed resource pool" — precisely the differentiation opportunity that distinguishes a decentralized compute network from single-machine solutions.

Among these, the quantitative effects of the GQA and low-precision inference mechanisms on VRAM footprint are shown in Figure 6-1 (the benefits of the remaining mechanisms are reflected in capacity ceilings and throughput improvements; see Chapter 5).

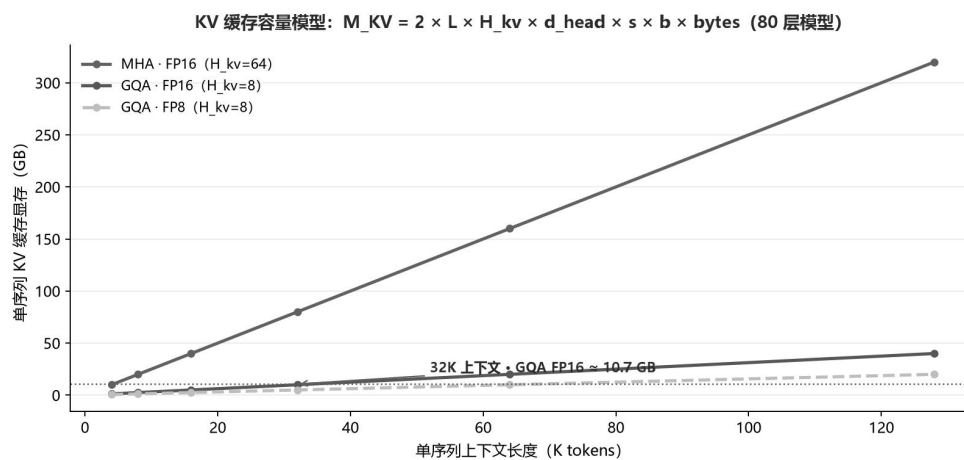


Figure 6-1 KV-cache capacity model: the compression effect of GQA and low precision on VRAM footprint (80-layer model, per sequence)

Figure 6-1 Data Recalculation Notes: the memory-compression comparison in this figure is supported by a unified parameter table, provided in Appendix C Table D-2 with this edition, including model tiers (7B / 70B class, each internally consistent) and context lengths, GQA group configurations, paged KV-cache block size and hit-rate methodology, pre- and post-compression single-sequence KV memory usage, and supportable concurrent sequence counts (listed separately for the two tiers); readers can independently recalculate accordingly; the MHA rows are comparison baseline rows without GQA compression. The hit rate uses a block-level methodology: within a calendar-day (UTC+0) statistical window, the number of reused KV physical-page reads ÷ the total number of KV physical-page reads; prefix-sharing reuse counts as a hit, and request-level or token-level statistics are not used. The values in the table are design estimates derived from the $\$6.4 \text{ M_KV}$ formula and are reviewed and updated quarterly as Phase 0 measurement samples expand.

6.5 Model Marketplace and Metered Settlement

- Billing by call count, token count, compute duration, or subscription; settlement is denominated in fiat or stablecoins by default, with the A12 Token serving as a fee-discount and incentive credential (see Chapter 8).
- Model developers share in call revenue; the revenue-split ratio is agreed in the model-listing agreement and publicly disclosed; metering records can be exported for audit.

Chapter 7 Layer 4: Agents and Applications (AaaS)

7.1 Visual Agent Orchestration Canvas

For users without engineering development capabilities, a low-code, drag-and-drop agent orchestration system is provided:

- **Configurable nodes:** trigger-type (Webhook, scheduled, event listener, dialog box), inference-type (invoking Layer 3 models or external models, with real-time adjustable parameters), tool-type (code execution, SQL query, external API, static analysis), and control-type (branching, loops, parsing, failure recovery and compensation).
- **Workflow DSL compilation:** the front-end canvas renders a directed graph; the compiler validates connectivity and circular dependencies, then converts it into a declarative workflow definition (JSON/YAML), stored as a versionable asset.
- **DAG execution engine:** a stateless back-end execution engine schedules nodes in topological order; dependency-free nodes execute concurrently, while dependent nodes receive intermediate results through a context state tree; execution records are retained in full for dispute replay.

The mathematical properties of execution scheduling are as follows: once a workflow is compiled into a directed acyclic graph $G = (V, E)$, the engine performs topological sorting using Kahn's algorithm — maintaining an in-degree table for all nodes, nodes with zero in-degree enter the ready queue and execute concurrently, and each completed node decrements the in-degree of its successors by one, with overall complexity $O(|V| + |E|)$; when the graph contains a cycle, some nodes' in-degree never reaches zero, which lets the compiler reject circular dependencies at submission time rather than failing at execution time. Scheduling parallelism is bounded by the maximum antichain width of the graph (in a graph of width w , at most w nodes can run concurrently at any moment), and the completion time of the entire workflow has a lower bound: the critical-path length, i.e., the longest weighted path from source to sink, computed by weighting each node's estimated duration. Based on this, the orchestration canvas shows users critical-path highlighting and an estimated completion window, turning "which nodes are slowing down the whole workflow" from empirical judgment into a computable conclusion.

Reference implementation of DAG topological sorting (Kahn's algorithm):

```
def kahn_toposort(graph: DAG):
```

```

in_deg = {v: 0 for v in graph.nodes}
for u in graph.nodes:
    for v in graph.edges[u]:
        in_deg[v] += 1
queue = [v for v, d in in_deg.items() if d == 0]
topo = []
while queue:
    v = queue.pop(0)
    topo.append(v)
    for w in graph.edges[v]:
        in_deg[w] -= 1
        if in_deg[w] == 0:
            queue.append(w)
if len(topo) != len(graph.nodes):
    return topo
raise CycleError

```

- **Tool-protocol standardization (forward-looking):** tool-type nodes will progressively align with open tool-integration protocols such as MCP (Model Context Protocol), so that external tool vendors can "adapt once, run on multiple platforms"; among multi-Agent workflows, task delegation and state mutual-recognition based on A2A (Agent-to-Agent)-style protocols will be explored, reserving standard interfaces for cross-organization agent collaboration. All of the above are design goals, subject to the maturity of the actual protocol versions.

7.2 Developer Toolchain: VS Code Plugin and Web IDE

- **Code-assistance plugin:** an editor extension based on the Language Server Protocol (LSP), connected to the Layer 3 inference gateway, providing context-aware (Fill-in-the-Middle) completions, documentation comments, and unit-test generation suggestions while code is being written.
- **Built-in static checks:** the plugin embeds a lightweight AST parser for real-time scanning, giving inline warnings and one-click fix suggestions for common dangerous patterns; like model outputs, the warning rules are positioned as auxiliary reference and do not replace manual audits or formal audit reports.
- **Cloud-based Web IDE:** the console embeds a browser-based IDE that can launch an isolated container environment with one click to run compilation and tests, reusing the same set of isolation and quota mechanisms as the §5.6 sandbox runtime.

7.3 Prebuilt Industry Agent Matrix

The platform plans to prebuild three categories of commercially ready-to-deploy Agent templates (launch plan: Phase 3 in Chapter 9), all implemented with the §7.1 orchestration system and backed by the Chapter 6 model services as the inference backend:

1. **Code and security assessment assistant: a multi-role collaborative workflow** — protocol structure analysis, test-case generation based on public vulnerability data, in-sandbox dynamic verification, and structured result summarization. The workflow's underlying layer supports mainstream reasoning patterns such as ReAct and CoT, along with model routing, automatically balancing cost and accuracy according to input complexity. The output is an internal reference report without formal CVSS ratings; formal audits are issued by qualified partners (not yet determined; to be disclosed once confirmed).
2. **On-chain data research Agent:** monitors DEX trading volumes, lending-protocol TVL anomalies, and whale-address movements, and periodically produces quantitative research reports combined with time-series analysis; data sources and methodologies are publicly verifiable. Research-type Agents access only public data sources holding redistribution rights; license agreements are completed before integrating commercial data providers, and the data-source list and authorization status are published with the product documentation.

3. **Enterprise knowledge-base Q&A Agent: vector database + hybrid retrieval + multi-tenant isolation, serving FAQ and knowledge Q&A for enterprises and communities; answers come with provenance tracing, and questions whose tracing fails are handled as "refusals to answer".**

7.4 Open Platform and Unified Cashier

- **Open API Gateway: Provides encrypted API access to third-party SaaS providers (IDE vendors, security tool vendors, data terminals, etc.), supports permission and quota management by call volume, concurrency, and seats, and provides billing detail export.**
- **Hybrid acquiring: the unified checkout currently accepts stablecoins (USDT/USDC); fiat payment channels will be opened only after the necessary licenses are obtained and the acquiring entity and payment service provider are determined and published—until then, the unified checkout does not offer fiat deposit channels in parallel. Once opened, tenants may choose between fiat and stablecoin channels as needed; all transactions are uniformly metered and reconciled by the platform. Fiat acquiring involves payment services and fund-custody arrangements, and follows the same principle as §2.4: the corresponding business is not conducted before the necessary licenses are obtained, and the acquiring entity and payment service provider will be published through official channels once determined. Expressions such as on-chain automatic conversion and drip dividends are no longer used—on-chain settlement serves only as an optional capability of the §8.6 protocol layer.**

Fiat-acquiring executability arrangement: before the acquiring entity and payment service provider are determined and published, the platform does not open fiat deposit channels, and the unified checkout accepts only stablecoin (USDT/USDC) settlement; the collection, refund, and reconciliation paths for enterprise customers are governed by the licensed acquiring entity and its terms of service as disclosed on the disclosure page. The determination and publication of the acquiring entity is the sole prerequisite for opening fiat channels to enterprise customers.

7.5 AaaS Settlement and Revenue Sharing

Settlement chain: user task → Agent service → model and tool calls → compute execution → unified metering → revenue shared among Agent developers, model providers, and compute providers according to contribution. Revenue-sharing rules are public, and records can be exported for audit.

Chapter 8 Token Economic Model

8.1 Design Principles

1. **Real revenue first: The anchor of platform value is service revenue, not Token circulating market capitalization.**
2. **Fixed total supply: A12 has an initial issuance of 100,000,000 tokens and will never be additionally issued.**
3. **Separation of incentives and settlement: Service settlement is denominated in fiat/stablecoins; Tokens are used for incentive distribution, fee discounts, and governance proposals, and tenants are not forced to hold tokens.**
4. **All mathematics is verifiable: Every mechanism in this chapter comes with constraints and examples; there are no hidden contradictions dependent on "undisclosed parameters".**

8.2 Token Allocation Model

Allocation Target	Ratio	Amount	Release Rules
-------------------	-------	--------	---------------

Ecosystem Building	60%	60,000,000	Fully locked in the early stage and excluded from circulation; subsequent release and allocation will be decided by DAO vote, with the resolution rules and results executed after publication on the official website
Community Airdrop and Early Incentives	30%	30,000,000	20% (6,000,000 tokens) unlocked in the first phase at launch; the remaining 80% (24,000,000 tokens) released linearly over 2 years from launch (12,000,000 tokens per year)
Liquidity and Market Making	10%	10,000,000	Injected at launch and replenished according to liquidity governance rules
Team and Operations	0%	0	No reserve; team and operating expenses are borne by the Operating Entity with its own funds

Note: this allocation plan is final. The team's zero allocation is an established platform decision, replaced by the Operating Entity funding long-term operations with its own capital. The ecosystem pool (60%, 60,000,000 tokens) is fully locked in the early stage, excluded from circulating supply, does not participate in the release curve of this table, and is separately listed as "locked and non-circulating" in supply disclosures; its subsequent release and allocation will be decided by DAO vote (§8.7), with the resolution rules and results executed after publication on the official website. Before an ecosystem-pool unlock resolution, the total circulating cap of the Token is the community airdrop (30%) plus liquidity and market-making (10%), i.e., 40% in total (40,000,000 tokens).

Explanation of the incentive mechanism for the team's 0% reserve: The team holds no Token share, which means team returns are completely decoupled from the secondary-market Token price—there is no team token-selling pressure, nor any proxy holding or dark pool (the team has no liquidatable Token position under its name, which can be jointly verified through on-chain transparency and third-party audits). The real return for team and operating expenses comes from the Operating Entity's equity appreciation and service revenue, not from Token liquidation. The cost of this design is a weakened link between team incentives and token price; therefore, revenue-linked buyback-and-burn (§8.3) converts service revenue growth into the common interest of all holders, forming indirect alignment.

Analytical and differential forms of airdrop circulating supply (t is the number of months after launch):

$$S_{\text{airdrop}}(t) = 6,000,000 + 24,000,000 \times \min(t/24, 1) \quad (\text{tokens}) \quad dS_{\text{airdrop}}/dt = 1,000,000 \text{ tokens/month} \quad (0 < t \leq 24; \text{ the derivative is zero after } t > 24)$$

The total circulating supply curve before an ecosystem-pool unlock resolution is shown in Figure 8-1—the early total circulating cap is 40% (40,000,000 tokens); the pace of supply expansion is mathematically capped, and buyback-and-burn further keeps actual circulation below the cap:

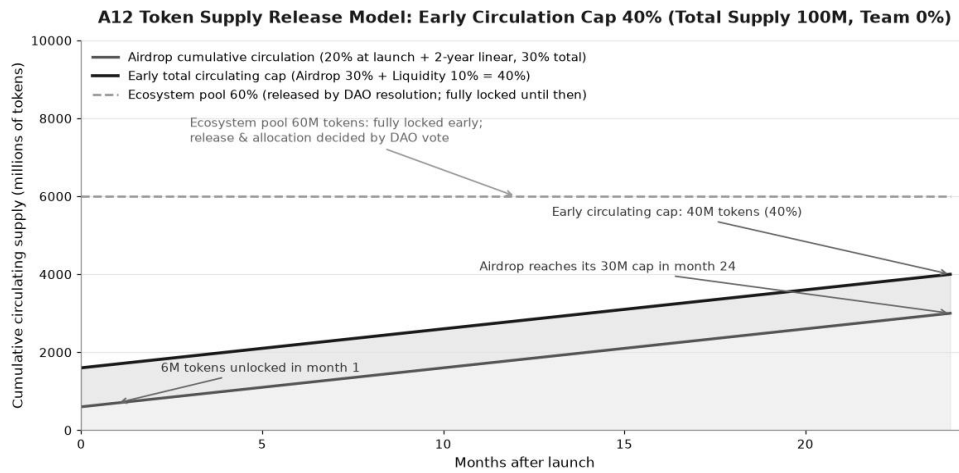


Figure 8-1 Token Supply Release Model: early circulation cap of 40% (the 60% ecosystem pool is released by DAO resolution; buyback-and-burn keeps actual circulation lower)

8.3 Buyback-and-Burn Mechanism

- **Burn source:** The platform uses 20% of the previous quarter's real net service revenue (initial ratio, adjustable via governance proposals, capped at 30%) to buy back A12 from the secondary market and burn it.

Audit caliber of net revenue (eliminating room for caliber manipulation):

$$NetRevenue_q = GrossRevenue_q - C_{compute_q} - C_{infra_q}$$

Here, $GrossRevenue_q$ is the platform's total service revenue for the quarter (all received revenue from CaaS/MaaS/AaaS); $C_{compute_q}$ is the cost of compute procurement and node revenue sharing; C_{infra_q} is the direct infrastructure cost of the validation cluster, bandwidth, etc. Personnel, marketing, and administrative expenses are not included as deductions (to prevent operating losses from being passed off as "low net revenue"). Each quarter, the three figures are published together with the burn transaction after an attestation report is issued by a third-party accounting firm. The attestation firm is selected according to the following criteria: membership in one of the Big Four international networks or equivalent scale, audit experience with Web3 or technology enterprises, and no independence issues with the Operating Entity or its affiliates; the engagement is targeted to be completed, and the firm's name disclosed through official channels, before the public beta launches.

- **Hard cap:** the quarterly burn must not exceed 100% of that quarter's buyback budget (i.e., only tokens actually bought back in the quarter may be burned; using reserves or borrowed funds to execute burns is not allowed); and when the remaining ecosystem co-building pool falls below 5,000,000 tokens, burns are suspended and the bought-back portion is injected into the ecosystem pool instead, ensuring the long-term incentive pool is not drained (this threshold is a long-term protective clause; during the public beta and early operations, "quarterly burn $\leq$ that quarter's buyback budget" is the binding constraint).
- **Example calculation:** Suppose the platform's net service revenue in a given year is USD 2,000,000; the buyback budget for that year is then USD 400,000, executed quarterly, purchased at market price, with burn transactions published on-chain. Burn volume is entirely determined by real revenue—if revenue is zero, the burn is zero, and there is no incentive to "dump reserves to sustain a deflationary narrative".

Accounting formulas for buyback and circulating supply. The quarterly buyback budget and burn quantity are defined as:

$$B_q = p_{\text{burn}} \times \text{NetRevenue}_q \text{ (} p_{\text{burn}} \text{ initially 20\%, capped at 30\%)} \quad \Delta S_q = B_q / \text{VWAP}_q$$

Where VWAP_q is the volume-weighted average price for the quarter, and ΔS_q is the burn quantity for that quarter—execution is volume-weighted rather than based on a single point-in-time price, suppressing room for timing and manipulation. Burn transactions are published on-chain, and anyone can independently verify via the three-stage chain of "net revenue evidence → buyback transaction → burn transaction".

Circulating supply curve. After the airdrop pool launches, cumulative circulation follows the release formula in §8.2 (6 million tokens in the first month, then a linear increase of 1 million per month, reaching the 30-million cap in month 24); the 60% ecosystem pool is fully locked before a DAO unlock resolution (§8.2), the early circulating cap is 40%, the pace of expansion is mathematically capped, and this is directionally consistent with the deflationary effect of buyback-and-burn. The formula above keeps "release pace — sell-pressure management—revenue buyback" independently accountable, avoiding any hidden contradiction that depends on unpublished parameters.

8.4 Airdrop and Points

Public beta and early contributions are recorded as points, which are converted into Token airdrops according to public rules. Points accrue based on effective workload (completed tasks, verification pass rate, online duration) and are decoupled from mere account counts; 20% of the airdrop pool is unlocked at launch, with the remaining 80% released linearly over 2 years (see §8.2), avoiding concentrated selling pressure at launch. Public-beta measurement commitment: the points-measurement methodology shares the same source as the post-launch CU-duration settlement methodology—the conversion factor may be adjusted, but the measurement methodology remains unchanged, and the conversion principles are published before the public beta begins. The settlement currency, paying entity, and settlement cycle for the public beta will be published before the beta begins ("settlement punctuality" in §9.2 refers to the proportion of payouts made in full on the published cycle). For high-value tasks, an optional USDT credit deposit (redeemable at any time; redemption resets reputation) provides a lower bound on the cost of misbehavior during the public-beta window without relying on Token staking.

Constraints on conversion coefficient adjustment: Adjustments to the point conversion coefficient are subject to three constraints—a single adjustment may not exceed $\pm 20\%$; the interval between two adjustments may not be less than 90 days; any downward adjustment must be announced on the disclosure page 30 days in advance with reasons stated. All adjustment records are archived and retrievable, and the adjustment procedure and caps are included in the disclosure scope of the Governance Charter (§8.7), ensuring that "unchanged measurement caliber" does not equal "the exchange level can be unilaterally lowered".

Determination of the first conversion factor: the three constraints above ($\pm 20\%$ / 90 days / 30 days' notice) govern the "adjustment of an existing factor"; the first factor is determined under separate rules—determination principle: anchored to the conversion benchmark between the public-beta points-measurement methodology and the post-launch CU-duration settlement methodology, i.e., the first factor is the reciprocal of "the equivalent CU-hours per unit of effective workload under both methodologies," so that the relative position of points holders is not changed by the

The public beta and airdrop are conditional on the licensing status and securities law opinion described in § 2.4; specific dates will be announced separately once satisfied: an open network public beta without incentives (involving no Token distribution, only validating network and product capabilities) may proceed before licenses are obtained; incentive activities involving Token airdrops will start after the necessary licenses are obtained and the legal opinion review is completed. Total incentive budget (executed upon launch of the incentive activity): 30,000,000 A12 (i.e., the full airdrop pool, distributed strictly following the § 8.2 release pace: 6,000,000 unlocked in the first phase, the remaining 24,000,000 released linearly over 2 years) + 5 BTC + 900,000 USDT, all borne by the Operating Entity (the platform) with its own funds; activity rules, participation eligibility, total rewards, and claim conditions are published on the platform's official website. Token rewards are subject to the § 8.2 release rules; fiat and BTC rewards are counted in the operating budget with a total cap, and do not conflict with the "separation of incentives and settlement" principle.

Public chain and DEX: the project has completed the self-development of a public chain and a decentralized exchange (DEX); this document does not elaborate on them. The protocol layer for settlement and attestation currently deploys smart contracts on mature public chains/L2s: the contracts handle settlement-credential attestation, burn publication, and (where regulation permits) stablecoin settlement channels; the introduction of the self-developed public chain and DEX is premised on regulatory assessment and confirmation of the compliance path, and the relevant arrangements will be published on the official website once determined. Token liquidity is provided by professional market-making institutions in the traditional manner, following open and transparent market practices: initial liquidity at launch is injected from the \$8.2 liquidity and market-making pool (10%); the names of specific market-making institutions and cooperation terms will be announced on the official website one week before launch.

```
contract A12BurnLedger {          using SafeERC20 for IERC20;          IERC20 public immutable A12;
address public constant BURN_SINK = 0x0000000000000000000000000000000000000000000000000000000000000000;          uint256 public
totalBurned;          event Burned(uint256 indexed quarter, uint256 amount, bytes32 revenueDigest);
function burn(uint256 amount, bytes32 revenueDigest, bytes calldata sig) external
{          require(_verifyAuditorSig(revenueDigest, amount, sig), "invalid attestation");
totalBurned += amount;          A12.safeTransfer(BURN_SINK, amount); // SafeERC20 safe transfer; on-chain
publication: anyone can independently verify the burn          emit Burned(currentQuarter(), amount,
revenueDigest);          } }
```

```
contract A12EscrowAndSlash {      struct Operator { uint256 locked; uint256 released; uint256 reputation;
```

```

}      mapping(address => Operator) public operators;          function lockEarnings(address op,
uint256 amount) external onlySettlement {                      operators[op].locked += amount;          // Earned →
Pending status recorded on-chain      }                      function slash(address op, uint256 amount, bytes32
evidenceDigest) external onlyArbiter {                          Operator storage o = operators[op];          uint256
penalty = amount < o.locked ? amount : o.locked; // inline min: slashing cap = escrowed amount
o.locked -= penalty;          o.reputation = (o.reputation * 8) / 10; // reputation decays directly
after default      emit Slashed(op, penalty, evidenceDigest);          }      }

```

Security audit plan: Before the above contracts are deployed in Phase 4, a complete audit must be completed by an independent third-party security firm—the selection criteria are consistent with the assurance firm in § 8.3 (membership in an international network or Web3 contract audit qualifications, and no independence issues with the Operating Entity); the full audit report is made public; before the audit is passed, the contracts are not deployed on mainnet and are not connected to real fund custody and slashing processes.

8.7 Governance

Platform operating decisions (scheduling strategy, pricing, risk control rules) are the responsibility of the Operating Entity; ecosystem governance is conducted through the A12 Global Governance DAO.

The DAO is open to community contributors, industry creators, technical contributors, and outstanding Workers, with 300 global governance seats in total and 40 seats opened in the first phase (subject to the official recruitment announcement).

Governance mechanism (design-target caliber; parameters are subject to the Governance Charter publication): Members are selected through "self-nomination + community review and publication", with a 12-month term renewable once; proposals must be co-signed by no less than 10% of members to enter a vote; general matters are passed by simple majority with a quorum participation rate of no less than 25%; major matters such as allocation parameter adjustments and slashing rule changes require no less than a two-thirds majority; resolutions are executed by the Operating Entity within a time limit and the execution results are published on the official website; members who are absent from votes three consecutive times without cause or who violate conflict-of-interest recusal rules are removed. The technical vehicle adopts off-chain snapshot voting + on-chain publication and attestation (consistent with the §8.6 protocol layer, without relying on a self-built public chain).

DAO member rights:

1. Participate in ecosystem proposals, voting, and governance of major matters;
2. Participate in platform rules and related governance decisions;
3. Receive corresponding incentives based on ecosystem contributions;
4. Enjoy dedicated points and related ecosystem benefits;
5. Prioritized participation in A12 global ecosystem building.

Boundary of powers and responsibilities between the DAO and the Operating Entity (eliminating the tension of "centralized operations vs. DAO"): The DAO is the ecosystem governance layer, not the platform operations layer. Ecosystem matters are substantively decided by the DAO; platform operating matters are the responsibility of the Operating Entity, which accepts DAO recommendations. The boundary is as follows:

Matter Category	DAO Authority	Operating Entity Role
Ecosystem incentive distribution, ecosystem activities, community proposals	Proposal right + voting right; resolutions are binding on the use of the Ecosystem Pool	Execute resolutions and publish; if not adopted, a written explanation of reasons must be provided within 14 days
Scheduling strategy, pricing, risk	Recommendation right (no voting)	Final decision-making authority; the

control, and compliance rules	binding force)	adoption of DAO recommendations is published regularly
Token economic parameter adjustments (e.g., moving the buyback ratio within the 20%–30% range)	Proposal right + voting right; resolutions are submitted to the Operating Entity for execution	Execution is subject to the hard caps of §8.2 and §8.3 and may not exceed them

This division is consistent with the §2.1 positioning of "centralized operations + distributed execution": the platform's operating responsibility entity is single and clear (avoiding decorative governance that "does not deliver on its words"), while the allocation rights of ecosystem resources are substantively handed over to the DAO (avoiding the appearance of "pseudo-decentralization").

Governance anti-capture mechanisms (design targets; details subject to the Governance Charter as published):

① **dual proposal threshold** — a proposal must simultaneously satisfy a seat co-sponsorship requirement of $\geq 10\%$ and a minimum token-holding/reputation threshold for the proposer, preventing low-cost collusion among individual seats; ② **per-proposal cap** — any proposal to use the Ecosystem Pool may deploy no more than 5% of the pool's unreleased balance, and oversized items must be split into separate votes; ③ **conflict-of-interest recusal** — proposers and their affiliates may not vote on proposals from which they benefit; violations render the proposal invalid and trigger removal procedures (the definition of "conflict-of-interest recusal" is written into the Governance Charter); ④ **emergency pause** — upon discovering a non-compliant or malicious proposal, the Operating Entity may suspend execution for up to 14 days and publicly disclose the reasons; if execution is not resumed within that period, the matter must be submitted to a full DAO vote; ⑤ **uniform attestation standard** — Ecosystem Pool disbursements and the §8.3 buyback-and-burn are subject to the same set of third-party accounting attestation and disclosure standards; ⑥ **first cohort first** — the selection criteria and elected roster of the first 40 seats are published before DAO launch.

Governance effective-document list: (1) the Governance Charter (including selection rules, voting rules, conflict-of-interest recusal, and emergency-suspension procedures); (2) the first 40-seat roster and selection criteria (the same matter as anti-capture mechanism (6) "first roster first," unified timing: published before DAO launch, no longer listed separately as a duplicate item); (3) the interpretation rule for the per-proposal cap on ecosystem-pool amounts. Determination of "effective upon publication": the effective time is jointly determined by the publication page's timestamp (UTC) + the content's SHA-256 hash, both archived together with the content; the verifying party is any third party capable of independent verification—the timestamp is provided by the third-party timestamp mechanism described in §2.2 (RFC 3161 tokens / public Git commit hashes / on-chain hashes of the §8.6 attestation contracts), sharing the same attestation and archiving process as that mechanism; version-archiving rule: all historical versions are permanently retained in the /archive directory, and in case of dispute, the complete version with the earliest timestamp and matching hash in the archive prevails. The ecosystem pool (60,000,000 tokens) is fully locked in the early stage, and its release and allocation are decided by DAO vote (§8.2); before the Governance Charter is published, the ecosystem pool does not use any allocation; the target publication time for the Governance Charter is before October 31, 2026 (a target, not a committed date; aligned with the Phase 0 target launch window of early October 2026), with the responsible role being the Legal & Compliance Lead; if the charter is still unpublished past that time, the ecosystem pool remains fully frozen, and the Operating Entity must publish the reason for the delay and a revised timeline within 5 business days after the deadline; upon two consecutive delays, an "alternative authorization mechanism for the event the charter remains long-unpublished" (e.g., authorizing a DAO preparatory committee to vote first on the ecosystem pool's release and allocation) must be submitted to the governance-proposal process for deliberation—before such an alternative mechanism is adopted, the ecosystem pool does not use any allocation. All

parameters of anti-capture mechanisms (1)–(6) are subject to the published version of the charter.

Chapter 9 Technology Roadmap

9.1 Resource Focus Principle

Core resources are concentrated on four product lines: the scheduling platform (Layer 1/2), the tenant console and sandbox runtime, model services (MaaS, including the first vertical model pipeline), and the AaaS toolchain (orchestration canvas + developer tools + pre-built Agents). All other directions (deep cross-vendor adaptation, protocol-layer deepening, additional vertical-industry models) are long-term experimental items and must pass the exit criteria of the preceding phase before entering the roadmap.

9.2 Phased Roadmap (with Exit Criteria)

Phase	Time	Delivery Targets	Exit Criteria (Go/No-Go)
Phase 0: Closed Beta	Early October 2026	Invitation-only 50–100 NVIDIA-stack nodes; core scheduling and five-layer verification online; tenant console (Web terminal + file channel); real settlement (fiat/stablecoin)	Verification pass rate $\geq 99\%$; scheduling success rate $\geq 95\%$; instance cold start ≤ 10 seconds; zero major security incidents
Phase 1: Global Public Beta · Worker Recruitment · Airdrop Round 1	From late October 2026	Open registration for the elastic best-effort mode (target: 1,000 nodes); deferred earnings release pool online; OpenAI-compatible gateway + continuous batching; Airdrop Round 1 and public-beta incentives are conditioned on the §2.4 licensing status and executed per official-site disclosures	Monthly node retention $\geq 60\%$; task dispute rate $< 1\%$; on-time settlement rate 100% (proportion of full payouts made within the published settlement cycle); median TTFT $\leq 400\text{ms}$ ($\leq 8\text{K}$ context tier; targets for the 32K long-context tier set separately)
Phase 2: SLA Commercialization	2027 H1	Dedicated guaranteed mode + redundant deployment; MaaS online; first vertical model pipeline delivered in Beta; first enterprise tenants	SLA target availability achieved; at least 3 paying enterprise tenants; positive gross margin (gross margin composition accounted for and disclosed separately by compute services and model revenue); GPU effective utilization $\geq 75\%$ (continuous batching tier)
Phase 3: AaaS and Expansion	2027 H2	Orchestration canvas + Web IDE public beta; pre-built Agent matrix online; experimental ROCm support; buyback-and-burn initiated	AaaS monthly revenue > 0 and growing; ROCm task pass rate $\geq 90\%$
Phase 4: Protocol Layer	2028+	Settlement evidence-deposit contracts on mature public chains; cross-vendor adaptation research	Regulatory assessment passed; audit reports made public

Note: Phase 0 closed beta (no Token distribution) is not committed to a calendar date—its launch is conditioned on §2.4 license-status confirmation and team capacity readiness; subject to these prerequisites

being met, the target launch window is early October 2026 (a target window, not a committed date), and the actual launch is subject to acceptance publication. Airdrops and incentive activities are conditioned on §2.4 license status and securities-law opinions (see §8.5), with dates announced separately once satisfied. The timeline is driven by engineering acceptance—phases may be postponed but never skipped: any unmet Go/No-Go criterion postpones expansion, and no phase is skipped at the cost of overextending the team (see §5.3 for the first-principles basis of the criteria—the supply-side economics framework).

9.2.1 Acceptance Report Specification (Fixed Annex)

An acceptance report is publicly released at the end of each phase. The report consists of two parts: "nine fixed fields + phase-specific additional fields"; the fixed fields do not change across phases: ① node scale (registered/active/qualified node counts, broken down by resource tier; including cross-identity deduplication statistics — declared device count / deduplicated device count / difference, corresponding one-to-one to the globally unique serial-number mutual exclusion in §4.5); ② task volume (total tasks in the period and distribution by task type); ③ verification pass rate (compiled per the §4.2 L2 and L4 review criteria); ④ scheduling success rate (proportion of first-attempt scheduling successes, distinguishing elastic best-effort from dedicated guaranteed modes); ⑤ instance cold-start P50/P95; ⑥ TTFT P50/P95 ($\leq$ 8K context tier; the 32K long-context tier listed separately); ⑦ GPU effective utilization (continuous batching tier, P50); ⑧ incident list (incident severity, duration of impact, root cause, and remediation items); ⑨ unmet items and reasons (item-by-item comparison against the phase's Go/No-Go criteria, listing deviations and attributions). Phase-specific additional fields are listed by each phase in its launch announcement and may only be added, not removed — fixed fields may not be deleted or replaced. The first acceptance report is published within 7 days after the end of Phase 0, and thereafter within 7 days after the end of each phase; the sole condition for entering the next phase is that all criteria of the preceding phase are met and the acceptance report is published (no commitment to a calendar date). This exclusivity statement admits no exception for fundraising progress, market windows, or staffing arrangements.

9.3 Frontier Technology Outlook (Research Horizons)

Positioning statement: This section depicts A12's medium- to long-term technology vision, intended to illustrate the platform's evolution direction and technical taste. All items below are research items or long-term outlooks and do not constitute delivery commitments for any phase; before any item enters the formal roadmap, it must pass the quantitative exit criteria of its preceding phase (see §9.2) and be announced separately.

A12's long-term blueprint can be summarized in one sentence: enable every idle GPU in the world to participate in the AI value chain through a unified trusted interface, and make agents the native consumers of compute. Along the vertical value chain of "CaaS → MaaS → AaaS", technological evolution unfolds across three horizons.

Focus note: This chapter deliberately retains only directions directly related to the platform's core capabilities (scheduling, inference, Agent execution); longer-term or cross-domain directions such as federated learning, differential privacy, homomorphic encryption, model watermarking, carbon-aware scheduling, WASM sandboxes, and post-quantum cryptography are not covered here and will be discussed separately in

subsequent technical blogs as vertical-industry expansion proceeds — avoiding the impression of "wanting to do everything, doing nothing in depth".

Horizon One: Compute Layer (present → 2027) — making distributed compute "trustworthy, usable, and schedulable".

Frontier Direction	Key Concept	Relationship to the Platform
eBPF kernel-level telemetry	Safely run sandboxed probes in the Linux kernel to collect system calls, network, and GPU driver events at near-zero overhead	Upgrades L3 runtime telemetry from user-space sampling to kernel-level observation, compressing the room for spoofing
Intent-driven scheduling (Intent-based)	Tenants declare only objectives (SLO, budget, latency); the scheduler automatically solves for resource configuration and deployment topology	Evolves from "matching tasks to resources" to "objective-driven orchestration", a natural extension of the §5.3 scheduling model
Cluster digital twin	Build a digital mirror of the entire resource pool using discrete-event simulation; scheduling policies are replayed and validated on the twin before going live	Reduces the production risk of new scheduling policies and supports SLA capacity planning

Horizon Two: Model and Execution Layer (2027 → 2028) — deeply coupling frontier model efficiency with distributed execution.

Frontier Direction	Key Concept	Relationship to the Platform
FSDP / ZeRO sharded training	Shard optimizer states, gradients, and parameters across multiple devices, lowering the memory barrier of full-parameter fine-tuning to what a single card can bear	Expands the spectrum of training tasks the distributed resource pool can accommodate
FP8 mixed-precision training	Forward/backward passes executed in FP8, with critical accumulations retained in higher precision	Together with low-precision inference, forms the platform's "precision–cost" product curve
Speculative decoding / FlashAttention-3 / Disaggregated Serving	See §6.4 Performance Engineering Outlook	Continuous evolution of inference efficiency at the MaaS layer
TEE confidential computing	CC-mode GPU hardware-encrypted memory and computation	See §4.7; removes the trust bottleneck of "placing highly sensitive enterprise workloads on distributed resources"

Horizon Three: Agent and Economic Layer (2028+) — settlement infrastructure for the Agentic Economy.

When agents become the primary consumers of compute, the scheduling platform will evolve from "a cloud for humans" to "a compute marketplace for Agents":

- MCP / A2A tool and collaboration protocols (see §7.1): Agents discover tools, delegate tasks, and mutually recognize state through standard protocols;**

2. **Test-time compute scaling: through strategies such as CoT / ToT / multi-sample voting, let models "think a little longer" on hard problems, trading cost for quality; accordingly, the scheduling platform offers three tiers of inference service — "fast / standard / deep";**
3. **Agent autonomous settlement: agents autonomously procure compute (bidding, price locking, plans) under preset budgets and policies, with auditable settlement records — this connects with the smart-contract settlement capability of the §8.6 protocol layer;**
4. **Marketization of compute as a factor: reputation, tiers, SLAs, and real-time supply and demand jointly form a public compute price signal, evolving resource pricing from "posted pricing" to "market-based matching".**

Horizons One and Two above are engineering evolution paths; Horizon Three is a directional vision; the relationship among the three is one of layered dependence and stepwise validation — without a trustworthy compute layer there is no efficient execution layer, and without execution-layer scale there is no settlement demand for the agentic economic layer. All content in this section is subject to the forward-looking statements in Chapter 10.

Chapter 10 Risk Factors and Disclaimer

1. **Execution risk: platform delivery depends on the core engineering team being in place and stable; team disclosure information is in §2.2, and until disclosure is complete, the related risks are assessed by readers at their own discretion. Mitigation: the phased roadmap sets quantitative Go/No-Go exit criteria, and each phase publishes an acceptance report per the §9.2.1 "Acceptance Report Specification" (nine fixed fields + phase-specific additional fields, published within 7 days after the phase ends).**
2. **Technical risk: container isolation cannot eliminate all escape risks; the verification system cannot absolutely prevent overselling and can only suppress it through economic and operational means; cross-vendor adaptation has uncertain timelines; the corpus construction and training outcomes of the vertical model pipeline are uncertain. Mitigation: three layers of protection — container isolation + purpose review + abuse compensation fund (see §4.7); SLA active-standby redundancy and reputation-based admission suppress overselling (see §5.3, §4.5).**
3. **Market risk: compute price volatility, cloud vendor competition, and changes in AI workload structure may all affect the platform's revenue assumptions. Mitigation: dual-path scheduling routes tasks by their characteristics to the optimal path, and the revenue structure spans the CaaS/MaaS/AaaS three layers, reducing dependence on any single market (see §5.2, §3.1).**
4. **Regulatory risk: digital-asset-related activities face different requirements in different jurisdictions, and the functions available to the Token may be adjusted with regulation; hybrid acquiring and protocol-layer capabilities are premised on regulatory assessment. Mitigation: jurisdiction-based admission matrix and secondary verification at the claiming step (see §8.5); licensed businesses are not conducted before the license is obtained (see §2.4).**
5. **Smart contract risk: smart contracts may contain undiscovered vulnerabilities (e.g., reentrancy, integer overflow, permission misconfiguration); before the third-party security audit is completed, contract interactions carry the risk of fund loss. Mitigation: the contract serves only as an optional protocol-layer capability for settlement evidence deposit and burn disclosure (see §8.6); before deployment it must pass an independent third-party audit with the full audit report published, and before the audit it will not go on mainnet or touch real funds.**

6. **Forward-looking statements:** All metrics contained in this whitepaper are design targets and all roadmaps are plans; nothing constitutes any promise of returns. Actual results may differ materially from plans.
7. **Disclaimer:** This whitepaper does not constitute investment advice, a securities offering document, or legal opinion. Rights in third-party trademarks belong to their respective owners.

Appendix A Glossary

Term	Definition
Operator	The platform account principal (natural person or legal entity); the aggregation unit for reputation, settlement, and penalties
Resource Lease	A mutually exclusive resource lock record on the "device serial number + operator" dimension in the platform ledger
Random Beacon (VRF)	A verifiable random function used to generate unpredictable, ex-post verifiable challenge seeds
Deferred Earnings Release Pool	The three-state Earned → Pending → Released earnings flow and default slashing mechanism
Communication-to-computation ratio α_{comm}	The ratio of a task's cross-node communication time to pure computation time; the basis for dual-path scheduling
CU	Comprehensive node capability score, weighted from measured sub-metrics
Continuous Batching	An inference scheduling technique that dynamically inserts/removes requests at iteration granularity
HAL	Hardware adaptation layer, unifying the task submission interface and disclosing the backend adaptation matrix
CPT / SFT	Continued pre-training / supervised fine-tuning, the two training stages of the vertical model pipeline
DPO	Direct Preference Optimization, a model alignment method that builds preference pairs from tool execution results
TTFT	Time To First Token (TTFT), the core metric of online inference experience
Model Card	Model Card, a standardized document recording a model's intended uses, limitations, and evaluation results
EWMA	Exponentially weighted moving average, a statistical estimation method for telemetry baselines (see §4.2)
Hockney Model	Classic communication time model: $T \approx \text{fixed latency} + \text{data volume} / \text{bandwidth}$ (see §5.2)
GQA	Grouped-query attention, an attention structure that compresses KV cache memory usage by sharing key-value heads (see §6.4)

Critical Path	The longest weighted path from start to end in a DAG; the lower bound on workflow completion time (see §7.1)
Power of Two Choices	The Power of Two Choices load-balancing strategy: randomly pick two candidates and dispatch to the less loaded one (see §6.3)
Speculative Decoding	An accelerated generation method in which a small model drafts and a large model verifies (see §6.4, §9.3)
Prefill/Decode Disaggregation	An inference serving architecture that splits first-token generation and subsequent decoding onto different resource pools (see §6.4, §9.3)
TEE / Confidential Computing	Trusted execution environment with hardware-level encrypted memory and computation (see §4.7, §9.3)
eBPF	Programmable observability technology for the Linux kernel, used for kernel-level low-overhead telemetry (see §9.3)
MCP / A2A	Open tool-integration protocol for Agents / inter-agent collaboration protocol (see §7.1, §9.3)
PQC (Post-Quantum Cryptography)	A next-generation cryptographic algorithm system resistant to quantum-computing attacks. Not covered in this paper; terminology index only (see §9.3)
Digital Twin	A replayable-simulation digital mirror built for the physical resource pool (see §9.3)
Commit–Reveal	An anti-manipulation protocol paradigm in which a hash commitment is published first and the original value revealed later (see §4.2)
Erlang C / M/M/c	A multi-server queueing model used for concurrency capacity planning and latency back-solving (see §6.3)
Gas Metering	An Ethereum-style resource metering convention, used by A12 for task-level billing and auditing (see §3.4)
Collateral / Slashing	The industry-standard collateral-slashing mechanism, adopted by A12's slashing function (see §3.4, §4.5)
Chinchilla Scaling Law	The empirical scaling relationship of model loss with respect to parameter count N and training token count D (see §6.2)
Roofline Model	A peak-performance model under arithmetic-intensity constraints (see §6.4)
ReAct / CoT / ToT	Agent reasoning + action / chain-of-thought / tree-of-thought decision patterns (see §7.3, §9.3)
Model Routing	An inference strategy that automatically selects models of different sizes based on input

complexity (see §7.3, §9.3)

Federated Learning / Differential Privacy	Privacy-preserving distributed training and noised gradient aggregation. Not covered in this paper; terminology index only (see §9.3)
CaaS	Compute as a Service — the external commercial form of the platform's resource layer (Layer 1/2)
MaaS	Model as a Service — the external commercial form of the platform's model layer (Layer 3)
AaaS	Agent as a Service — the external commercial form of the platform's application layer (Layer 4)
SLA	Service Level Agreement — a quantitative commitment on availability, performance, and compensation
DAO	Decentralized Autonomous Organization (see §8.7 Governance Mechanism)
IDC	Internet Data Center
RAG	Retrieval-Augmented Generation
LoRA / QLoRA	Low-Rank Adaptation and its quantized version, parameter-efficient fine-tuning methods
MSB	Money Services Business, the U.S. FinCEN money services business license
SEC	U.S. Securities and Exchange Commission
DEX	Decentralized Exchange
NVML	NVIDIA Management Library, a GPU state telemetry library
WSS	WebSocket Secure, an encrypted WebSocket channel
SLO	Service Level Objective (the quantitative prerequisite metric of an SLA)
EIP-1559	Ethereum Improvement Proposal 1559, the fee pricing mechanism of base fee + tip (see §3.4)
Go/No-Go	The quantitative exit-criteria decision mechanism for each roadmap phase (see §9.2)

[Appendix B Index of Figures and Tables](#)

Figures: Figure 3-1 Four-Layer Architecture (§3.2); Figure 4-1 Five-Layer Verification System (§4.2); Figure 5-1 Dual-Path Routing (§5.2); Figure 5-2 Task Lifecycle State Machine (§5.5); Figure 6-1 KV Cache Capacity Model (§6.4); Figure 8-1 Supply Release Model (§8.2).

Main tables: Mandatory Disclosure Checklist (§2.2); Design Reference Frame (§3.4); Resource-Tier Scheduling Constraints (§5.4); Acceptance Report Fixed Fields (§9.2.1); DAO and Operating Entity Authority Boundaries (§ 8.7); Token Allocation Model (§ 8.2); Buyback-and-Burn Parameters (§ 8.3); Risk Factors (Chapter 10); Glossary (Appendix A); Figure Parameter Tables (Appendix C).

Appendix C Figure Parameter Tables (Figure 5-1 / Figure 6-1)

This appendix provides the recalculation parameter tables for Figure 5-1 and Figure 6-1. Corroboration path: the values in the tables are design estimates derived from the §5.2 Hockney communication model, Roofline throughput estimation, and the §6.4 M_{KV} formula; they are corroborated by the platform's validation cluster during the Phase 0 internal beta (50–100 invited NVIDIA-stack nodes, see §9.2) through measured recalculation of the five task types and two model tiers—measured T_{comm} / $T_{compute}$ / memory usage are compared item by item against this table, and the measurement records are archived on the disclosure page; any third party can also independently recalculate and compare using the parameters in this appendix. Versioning: the two parameter tables are maintained under version control, with each update labeled with a version number (e.g., D-1.1 / D-2.1) and a date stamp, and all historical versions retained in the /archive archive for citation and comparison; the parameter ranges are reviewed and updated quarterly as Phase 0 measurement samples expand. Table D-1 additionally provides two columns, T_{comm} (by bandwidth tier) and $T_{compute}$ (including hardware-tier assumptions), so that $\alpha_{comm} = T_{comm} \div T_{compute}$ can be recalculated tier by tier, rather than giving only the ratio without the denominator.

Table D-1 Figure 5-1 Parameter Table: α_{comm} estimation ranges (by bandwidth tier), T_{comm} / $T_{compute}$ values, and routing assignments

Task Type	Per-Step Synchronization Volume (Anchored)	T_{comm} (s/step, by bandwidth tier)	$T_{compute}$ (s/step, hardware tier)	α_{comm} Range (corresponding bandwidth tier)	Routing Assignment	Boundary-Task Measurement Points
Full-parameter fine-tuning (7B-class, data parallel)	≈ 28 GB/step	2,607 @100M; 261 @1G; 26 @10G; 9.5 @25G; 0.6 @IB400G	3–9 (RTX 4090 / H100-class single GPU, η_{eff} at 30–60% of peak, per §5.2)	0.1–870 (tier by tier: 290–870 @100M; 29–87 @1G; 2.9–8.7 @10G; 1.1–3.2 @25G; 0.1–0.3 @IB400G)	Centralized cluster— $\alpha_{comm} > 1$ across all bandwidth tiers except IB-class interconnects	7B full-parameter fine-tuning (28 GB/step synchronization volume)
Distributed pre-training / large-scale CPT (70B-class)	≈ 280 GB/step (scaling linearly with parameter count)	255 @10G; 25.5 @100G dedicated line; 6.2 @IB400G	20–60 (H100 $\times$ 8 cluster, η_{eff} 30–60%)	4–13 @10G; 0.1–0.3 @IB400G	Centralized cluster—even at the highest tier of the distributed resource pool (IDC 10 Gbps), α_{comm} already reaches 4–13, far above the threshold of 1; moreover, tensor/pipeline parallelism requires intra-node NVLink/IB (Tier-S, §5.4)	Sharded gradient synchronization for 70B-class CPT
LoRA / QLoRA fine-	≈ 0.1 –1 GB/step	0.9–9 @1G; 0.09–	3–9 (RTX 4090-	0.1–3 @1G; 0.01–0.3	Boundary zone: assignment	LoRA fine-tuning (< 1

Aelion Twelve (A12) Super-Orchestration Cloud Platform Technical Whitepaper · v1.0

tuning	(only adapter parameters are synchronized)	0.9 @10G	class single GPU)	@10G	determined by measured re-verification	GB/step)
Online inference ($\leq$ 8K context)	KB–MB scale (request and response)	10^{-3} – 10^{-2}	0.05–2 (single request, prefill + decode)	10^{-3} – 10^{-2}	Distributed resource pool	— (far from the threshold)
Embedding / Rerank / RAG and batch processing	KB–MB scale	10^{-4} – 10^{-2}	10^{-2} – 10^1	10^{-4} – 10^{-2}	Distributed resource pool	— (far from the threshold)
Routing threshold	—	—	—	$\alpha_{\text{comm}} = 1$; assignment on either side of the threshold is determined by measured re-verification	—	Boundary tasks are tiered by measured α_{comm} and measured bandwidth

Table D-2 Figure 6-1 Parameter Table: model tiers (7B / 70B class), GQA configurations, and KV memory usage

Model Tier	Context Length	GQA Grouping Configuration	Paging Block Size and Hit-Rate Basis	Per-Sequence KV VRAM Usage (before compression → after paging)	Concurrent 32K Sequences Supported
7B-class (32 layers, d_model 4096, d_head 128) · baseline row	32,768 tokens	MHA (H_kv = 32, no GQA compression)	Block size 16 tokens; hit rate (block-level, natural-day window) design estimate P50 $\approx$ 0.35	FP16: $\approx$ 17.2 GB → $\approx$ 17.2 GB (baseline row: no GQA compression; paging only eliminates fragmentation)	$\approx$ 3 sequences (80 GB single GPU – 7B weights 14 GB = 66 GB available)
7B-class + GQA	32,768 tokens	GQA 8 groups (H_kv = 8)	Block size 16 tokens; hit rate (block-level, natural-day window) design estimate P50 $\approx$ 0.35	FP16: $\approx$ 4.3 GB → $\approx$ 4.3 GB	$\approx$ 15 sequences (66 GB available)
7B-class + GQA + FP8	32,768 tokens	GQA 8 groups (H_kv = 8)	Block size 16 tokens; hit rate (block-level, natural-day window) design estimate P50 $\approx$ 0.35	FP8: $\approx$ 2.1 GB → $\approx$ 2.1 GB	$\approx$ 30 sequences (66 GB available; FP8 doubles concurrency)
70B-class (80 layers, d_model 8192, d_head 128) · baseline row	32,768 tokens	MHA (H_kv = 64, no GQA compression)	Block size 16 tokens; hit rate (block-level, natural-day window) design estimate P50 $\approx$ 0.35	FP16: $\approx$ 85.9 GB → $\approx$ 85.9 GB (baseline row: no GQA compression)	0 (not supported on a single GPU)
70B-class + GQA	32,768 tokens	GQA 8 groups (H_kv = 8)	Block size 16 tokens; hit rate (block-level, natural-day window) design estimate P50 $\approx$ 0.35	FP16: $\approx$ 10.7 GB → $\approx$ 10.8 GB (same configuration as the \$6.4 in-text example)	$\approx$ 1 sequence (dual-GPU server 160 GB – 70B weights 140 GB = 20 GB available)
70B-class + GQA + FP8	32,768 tokens	GQA 8 groups (H_kv = 8)	Block size 16 tokens; hit rate (block-level, natural-day window) design estimate P50 $\approx$ 0.35	FP8: $\approx$ 5.4 GB → $\approx$ 5.4 GB	$\approx$ 3 sequences (20 GB available)

Note: both MHA rows are comparison baseline rows with "no GQA compression"; the identical values before and after compression are expected behavior—the paging mechanism only eliminates VRAM fragmentation and performs no compression, and should not be misread as paging having no effect; the compression effect is reflected in the VRAM reduction of the GQA rows relative to the baseline rows (7B-class 17.2 → 4.3 GB, 70B-class 85.9 → 10.7 GB). The 70B-class tier uses the same configuration as the \$6.4 in-text example (80 layers, H_kv = 8, FP16 per-

sequence ≈ 10.7 GB); the 7B-class is a typical tier for single-GPU concurrency scenarios, and the two tiers correspond respectively to the two back-calculation objectives of "compression effect comparison" and "single-GPU supported concurrency."

References

The following are the key academic papers, technical standards, and official documents cited in this whitepaper. All mechanism naming and formula conventions in the main text follow the sources listed above; metrics labeled "design target" are defined by this whitepaper itself and do not cite any unverified third-party measurement data. The access date for online sources is uniformly 2026-09-14.

Academic Papers

1. Hoffmann et al., Training Compute-Optimal Large Language Models (Chinchilla scaling law, see §6.2), 2022, arxiv.org/abs/2203.15556arxiv.org/abs/2203.15556
2. Rafailov et al., Direct Preference Optimization: Your Language Model is Secretly a Reward Model (DPO, see §6.2), NeurIPS 2023, arxiv.org/abs/2305.18290arxiv.org/abs/2305.18290
3. Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models*, 2021, arxiv.org/abs/2106.09685
4. Dettmers et al., *QLoRA: Efficient Finetuning of Quantized LLMs*, 2023, arxiv.org/abs/2305.14314
5. Leviathan et al., Fast Inference from Transformers via Speculative Decoding (speculative decoding, see §6.4), ICML 2023, arxiv.org/abs/2211.17192
6. Ainslie et al., GQA: Training Generalized Multi-Query Transformer Language Models from Multi-Head Checkpoints (see §6.4), 2023, arxiv.org/abs/2305.13245arxiv.org/abs/2305.13245
7. Rajbhandari et al., *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*, SC 2020, arxiv.org/abs/1910.02054
8. Shazeer et al., Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer (MoE, see §6.4, §9.3), ICLR 2017, arxiv.org/abs/1701.06538arxiv.org/abs/1701.06538
9. Micikevicius et al., FP8 Formats for Deep Learning (see §9.3), NVIDIA, 2022, arxiv.org/abs/2209.05433arxiv.org/abs/2209.05433
10. Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness (see §9.3), NeurIPS 2022, arxiv.org/abs/2205.14135arxiv.org/abs/2205.14135
11. Shah, Bikshandi, Zhang et al., FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision (FlashAttention-3, see §9.3), 2024, arxiv.org/abs/2407.08608arxiv.org/abs/2407.08608
12. Williams, Waterman & Patterson, Roofline: An Insightful Visual Performance Model for Multicore Architectures (see §5.2, §6.4), Communications of the ACM, 2009, doi.org/10.1145/1506609.1506623doi.org/10.1145/1506609.1506623
13. Kahn, A.B., Topological Sorting of Large Networks (see §7.1), Communications of the ACM, 5(11): 558–562, 1962, doi.org/10.1145/368996.369025doi.org/10.1145/368996.369025
14. Erlang, A.K., Solution of some Problems in the Theory of Probabilities of Significance in Automatic Telephone Exchanges (M/M/c and Erlang C queueing models, see §6.3), Elektroteknikeren, 13, 1917

Technical Standards and Official Documents

1. [ERC-20 Token Standard, Ethereum Improvement Proposals \(see §8.6\)](#)
2. [OpenZeppelin Contracts security conventions \(SafeERC20 / Math\) \(see §8.6\)](#)
3. [Solidity official documentation \(see §8.6\)](#)
4. [Model Context Protocol \(MCP\) specification \(see §7.1, §9.3\)](#)
5. [NIST Post-Quantum Cryptography \(PQC\) Standardization Project \(see §9.3\)](#)
6. [WebAssembly \(WASM\)](#) WebAssembly (WASM) and eBPF (see §4.7, §9.3)[eBPF](#)

Design Reference Framework

1. [Ethereum Whitepaper](#) Ethereum Whitepaper—reference for the account system and Gas metering (see §3.4)